

MapacheSecure



Sistema Móvil Gamificado para la
Autorregulación Digital

Integrantes del grupo

- Fernanda Arraño
- Javier Vieytes
- Benjamín Castillo

Índice

Índice	2
1. Introducción	5
2. Contexto del proyecto	6
3. Descripción del problema u oportunidad	12
3.1. Descripción del Problema	12
3.2. Análisis de causas y efectos	12
3.3. Representación Gráfica del Problema	13
Diagrama de Ishikawa	13
Árbol del problema	14
4. Descripción del mercado objetivo	15
4.1. Definición de los Usuarios y Clientes	15
4.2. Necesidades y Dolores del Mercado	15
Perfil de usuario 1	16
Perfil de usuario 2	16
Stakeholders Internos	17
Stakeholders Directos	17
Stakeholders Indirectos	17
5. Objetivo general y objetivos específicos	18
5.1. Objetivo General	18
5.2. Objetivos Específicos	18
6. Situación inicial del cliente (AS-IS)	19
6.1. Descripción del Proceso Actual	19
6.2. Puntos de Dolor Identificados	19
6.3. Diagrama de Flujo del Proceso Actual (AS-IS)	20
7. Planificación	21
7.1. Metodología de Gestión: SCRUM y Trello	21
7.2. Organización del Equipo y Roles Críticos	21
7.3. Cronograma de Actividades (Carta Gantt)	22
8. Servicios Cloud a utilizar (IaaS / PaaS / SaaS)	23
9. Conceptualización (Solución propuesta – Versión preliminar)	24
Funcionalidades Principales:	24
1. Módulo de Intercepción y Bloqueo Inteligente:	24
2. Motor de Desafíos Multimodal:	25
3. Sistema de Economía de Fichas (Puntos y Recompensas):	27
4. Panel de Control Parental (Gestor de Reglas):	28
5. Sistema de Notificaciones y Reportes:	29
6. Arquitectura de Sincronización Cloud:	30
10. Alcance, supuestos y restricciones	31
Alcance del Proyecto	31
Exclusiones del Proyecto	31
Entregables	32

Supuestos	32
Restricciones	32
11. Metodología de desarrollo y gestión del proyecto	34
Metodología Scrum	34
Planificación, Seguimiento y Control Semanal	34
12. Documentos y diagramas de diseño de la solución	40
Modelo de datos (MER/DER) o esquema de BD (tablas, relaciones)	41
Mockups de la app: seleccione foto de perfil, revisión de evidencias enviadas	45
Diagrama de arquitectura (alto nivel).	46
13. Arquitectura de software y patrones de diseño	47
13.1. Arquitectura de N-Capas (Monolito Modular)	47
13.2. Patrones de Diseño Implementados	47
13.3. Justificación de la Arquitectura	47
13.4. Representación Visual de la Arquitectura	48
C4 Nivel 1 - Contexto del Sistema	48
C4 Nivel 2 - Contenedores	49
14. Tecnologías, lenguaje y dependencias (stack definitivo)	50
Stack Tecnológico Definitivo	50
Dependencias del Backend (requirements.txt)	50
Estándares de Código	50
15. Configuración de servidor de producción (paso a paso)	53
Plataforma Cloud	53
Paso a Paso — Backend en Render	53
Paso a Paso — Base de Datos en Supabase	53
16. Estado de avance del desarrollo (evidencia de producto funcional)	55
Funcionalidades Implementadas	55
Funcionalidades En Progreso	56
Calidad y Seguridad	56
17. Integraciones necesarias	57
17.1. Detalle de Integraciones Existentes y Planificadas	57
17.2. Consideraciones de Seguridad y Protección de Datos	58
17.3. Flujo de Datos y Comunicación entre Servicios	58
17.4. Evidencias de Pruebas de Integración	59
18. Estrategia y enfoque de calidad	61
19. Plan de pruebas	63
20. Tipos de pruebas aplicadas	65
20.1 pruebas unitarias	65
20.2 Pruebas de humo (smoke)	65
20.3 Pruebas de integración	65
20.4 Pruebas funcionales	65
20.5 Pruebas de regresión	66
20.6 Pruebas de rendimiento / carga	66
20.7 Pruebas de seguridad	66
20.8 Pruebas unitarias de backend	66

21. Herramientas y software utilizados	67
22. Ejecución de pruebas y resultados	69
Resultados por tipo de prueba	70
Pruebas unitarias (75 casos)	70
Pruebas de humo en dispositivo (9 casos)	79
Pruebas de recuperación 4 casos	80
Pruebas funcionales (4 casos)	81
Pruebas de regresión (4 casos)	81
Pruebas de carga / rendimiento (4 casos)	82
Pruebas de seguridad (4 casos)	83
Pruebas unitarias de backend (105 casos)	83
Conclusión 1	85
Referencias Bibliográficas	87
Anexos	88
Análisis estático de código — flutter analyze (Linter)	88

1. Introducción

En la actualidad, los dispositivos electrónicos forman parte central de la vida cotidiana de niños y jóvenes. Si bien estas tecnologías ofrecen oportunidades de aprendizaje y entretenimiento, su uso desmedido representa un desafío creciente para padres, educadores y profesionales de la salud. Las plataformas digitales están diseñadas para maximizar el tiempo que los usuarios pasan en ellas mediante estímulos visuales constantes, lo que dificulta que los niños desarrollen hábitos de uso equilibrado de manera autónoma.

En este contexto, se identificó una problemática concreta: los niños pueden llegar a pasar hasta 10 horas diarias frente a una pantalla, generando dependencia y limitando el tiempo dedicado a actividades esenciales para su desarrollo, como la lectura, el ejercicio físico o los hábitos de higiene. Frente a esto, surge *MapacheSecure*, una aplicación móvil Android que bloquea de forma inteligente las aplicaciones de entretenimiento de alto consumo, reemplazando ese tiempo con actividades interactivas y saludables. Al completarlas, los niños acumulan puntos que pueden canjear por recompensas definidas por sus padres o responsables, convirtiendo el autocontrol en una experiencia motivadora.

El presente informe describe el desarrollo de esta solución en detalle. Se expone la problemática detectada, los objetivos del proyecto, el alcance definido, los requerimientos funcionales y no funcionales, el diseño de la aplicación y las conclusiones del trabajo. El objetivo es ofrecer una visión completa del proceso de construcción de *MapacheSecure*, desde su concepción hasta su implementación técnica.

2. Contexto del proyecto

La masificación de los dispositivos móviles ha transformado profundamente la infancia. Hoy en día, los niños tienen acceso a smartphones y tablets desde edades muy tempranas, y aunque esto les abre puertas al aprendizaje y la creatividad, también los expone a un entorno diseñado para mantenerlos enganchados el mayor tiempo posible. Las plataformas de entretenimiento digital utilizan algoritmos de recomendación y sistemas de recompensa inmediata que resultan especialmente efectivos en cerebros en desarrollo, generando patrones de uso difíciles de controlar sin ayuda externa.

La idea detrás de MapacheSecure surgió de una observación cotidiana: en redes sociales es común ver videos donde niños reaccionan con crisis emocionales intensas al momento de que se les retira un dispositivo, lo que evidencia una dependencia que va más allá del entretenimiento. Esta situación motivó al equipo a buscar una solución que no solo limite el tiempo de pantalla, sino que convierta ese tiempo de desconexión en una experiencia positiva y formativa para el niño.

El público objetivo son niños de entre 5 y 15 años cuyos padres o tutores buscan herramientas concretas para establecer límites saludables sin generar conflictos. A diferencia de un simple bloqueo, MapacheSecure propone un sistema de desafíos y recompensas que hace que el niño participe activamente en la gestión de su propio tiempo.

En cuanto al estado del arte, existen aplicaciones que abordan parcialmente este problema. Forest permite establecer temporizadores de concentración usando la metáfora de plantar árboles. Freedom y Opal bloquean aplicaciones específicas por períodos definidos. StayFocusd limita el tiempo en sitios web. Sin embargo, ninguna de estas soluciones combina el bloqueo efectivo con actividades de reemplazo, un sistema de recompensas personalizable por los padres y una experiencia gamificada orientada específicamente a niños. La siguiente tabla resume esta comparación:

Forest

Es una aplicación de productividad que usa la metáfora de plantar un árbol para mantener el enfoque.



Imagen 1 "Interfaz de usuario y mecánica de gamificación ambiental de la aplicación Forest"

- **Funcionalidad:** Actúa como un temporizador de concentración donde el usuario "planta" un árbol que muere si sale de la aplicación.
- **Ventajas:** Posee una interfaz sumamente simple, intuitiva y visualmente atractiva, lo que facilita su adopción inicial.
- **Desventajas y Limitaciones:** Su principal debilidad es que no ejerce un bloqueo real sobre el dispositivo. Es una herramienta diseñada para la autogestión de adultos; un niño puede simplemente cerrar la app y el "castigo" de ver el árbol morir no es suficiente para frenar la dependencia.
- **Diferencia de MapacheSecure:** Nosotros aplicamos un bloqueo real del sistema y obligamos a completar actividades para desbloquearlo, no solo un aviso visual.

Freedom

Es una herramienta robusta diseñada para eliminar distracciones bloqueando el acceso a internet y aplicaciones específicas en múltiples dispositivos.



Imagen 2: "Interfaz de configuración de bloqueos y sesiones en Freedom"

Funcionalidad: Permite programar sesiones de bloqueo total para aplicaciones y sitios web seleccionados.

Ventajas: Es una solución multiplataforma (funciona en PC, Mac y móviles).

Desventajas: Es puramente restrictiva; al bloquear, no ofrece ninguna actividad alternativa al usuario, lo que genera frustración en niños.

Diferencia de MapacheSecure: A diferencia del "muro ciego" de Freedom, MapacheSecure propone actividades de reemplazo lúdicas que transforman la restricción en una oportunidad de aprendizaje.

Opal

Es una herramienta moderna enfocada en la productividad y en reducir el tiempo de exposición a pantallas mediante un enfoque de bienestar digital.



Imagen 3: "Dashboard de productividad y gestión de tiempo de la aplicación Opal"

Funcionalidad: Implementa bloqueos de aplicaciones específicos basados en horarios programados por el usuario.

Ventajas: Cuenta con una interfaz de usuario sumamente amigable, estética y moderna.

Desventajas: No está orientada al público infantil; su diseño y lenguaje están pensados para adultos que buscan mejorar su productividad laboral.

Diferencia de MapacheSecure: A diferencia de Opal, que es una herramienta de autogestión, MapacheSecure incluye un sistema de recompensas diseñado específicamente para motivar a los niños, haciendo que el cumplimiento de metas sea un juego y no solo una estadística de tiempo.

Stay Focused

Es una herramienta clásica de productividad que busca reducir la navegación excesiva limitando el tiempo que el usuario puede pasar en sitios web distractores.



Imagen 4: "Interfaz de configuración de límites de navegación en la herramienta Stay Focused"

Funcionalidad: Establece límites de tiempo diarios para sitios web específicos; una vez agotado el tiempo, el acceso se bloquea automáticamente.

Ventajas: Es extremadamente fácil de configurar y muy directa en su función.

Desventajas: Su enfoque principal es el navegador; no tiene un control integral sobre las aplicaciones instaladas en el sistema móvil, lo que permite al niño simplemente cambiar de app.

Diferencia de MapacheSecure: MapacheSecure ofrece un control total del dispositivo, no solo del navegador. Actúa sobre cualquier aplicación de entretenimiento, garantizando que el límite no se pueda evadir saltando de la web a una aplicación.

MapacheSecure

Tras analizar las soluciones actuales, se identifica una oportunidad para crear una herramienta que no sea puramente restrictiva ni fácilmente eludible por un menor.



Imagen 5: *"Identidad visual y concepto central del sistema MapacheSecure"*

- Qué hace: Implementa un sistema de bloqueo inteligente integrado con desafíos interactivos (cognitivos, físicos y de orden) y un catálogo de recompensas configurables.
- Ventajas: Es una solución integral y gamificada que no solo limita el tiempo, sino que educa y motiva al niño a realizar actividades saludables.
- Desventajas: Actualmente se encuentra en fase de desarrollo y prototipado técnico.
- Diferenciador principal: MapacheSecure transforma el concepto de "control parental" de una prohibición a un intercambio de valor positivo, donde el acceso al entretenimiento es una recompensa obtenida mediante el esfuerzo personal.

3. Descripción del problema u oportunidad

3.1. Descripción del Problema

Actualmente, los niños en edad escolar enfrentan una crisis de sobreexposición digital. El uso de dispositivos electrónicos ha dejado de ser una actividad complementaria para convertirse en el eje central del día a día, con promedios que superan las 10 horas diarias frente a una pantalla. Esta situación provoca una dependencia progresiva que desplaza actividades vitales como el juego físico, la lectura y la interacción social presencial.

El núcleo del problema no es solo el tiempo, sino la naturaleza del contenido: las aplicaciones de entretenimiento y redes sociales operan bajo una lógica de gratificación instantánea. Mediante algoritmos de retención diseñados para explotar la liberación de dopamina, los dispositivos capturan la atención del menor de forma compulsiva, dificultando su capacidad de autorregulación y generando una resistencia activa frente a cualquier intento de mediación parental.

3.2. Análisis de causas y efectos

Para comprender la profundidad de este fenómeno, es necesario desglosar los factores que lo alimentan y las consecuencias sistémicas que genera en el desarrollo infantil.

Análisis de Causas (Raíces del problema):

- **Diseño Afectivo y Adictivo:** Las plataformas digitales utilizan mecánicas de juego y recompensas variables que resultan especialmente efectivas en cerebros en desarrollo, los cuales aún no poseen la madurez cortical necesaria para el control de impulsos.
- **Ineficacia de herramientas actuales:** La mayoría de las soluciones de control parental son punitivas o meramente restrictivas. Al no ofrecer una alternativa atractiva, el bloqueo se percibe como una agresión, lo que fomenta el conflicto familiar.
- **Erosión de las rutinas analógicas:** La facilidad de acceso a la entretención digital ha desplazado las rutinas de hogar, haciendo que el esfuerzo requerido para leer un libro o hacer ejercicio parezca "aburrido" en comparación con la estimulación rápida de una pantalla.

Análisis de Efectos (Impacto en el menor):

- **Deterioro de la Salud Física:** El sedentarismo prolongado aumenta el riesgo de obesidad infantil y problemas posturales. Además, la luz azul de las pantallas altera los ciclos circadianos, provocando trastornos del sueño y fatiga crónica.

- **Dispersión Cognitiva:** La exposición a estímulos fragmentados y rápidos reduce la capacidad de atención sostenida, lo que se traduce en un bajo rendimiento escolar y dificultades para procesar información compleja.
- **Tensión en el Clima Familiar:** La falta de una herramienta mediadora convierte el límite digital en una fuente constante de peleas, debilitando la autoridad parental y el bienestar emocional del hogar.

3.3. Representación Gráfica del Problema

A continuación, se presentan los diagramas técnicos que sintetizan la relación entre los factores antes mencionados.

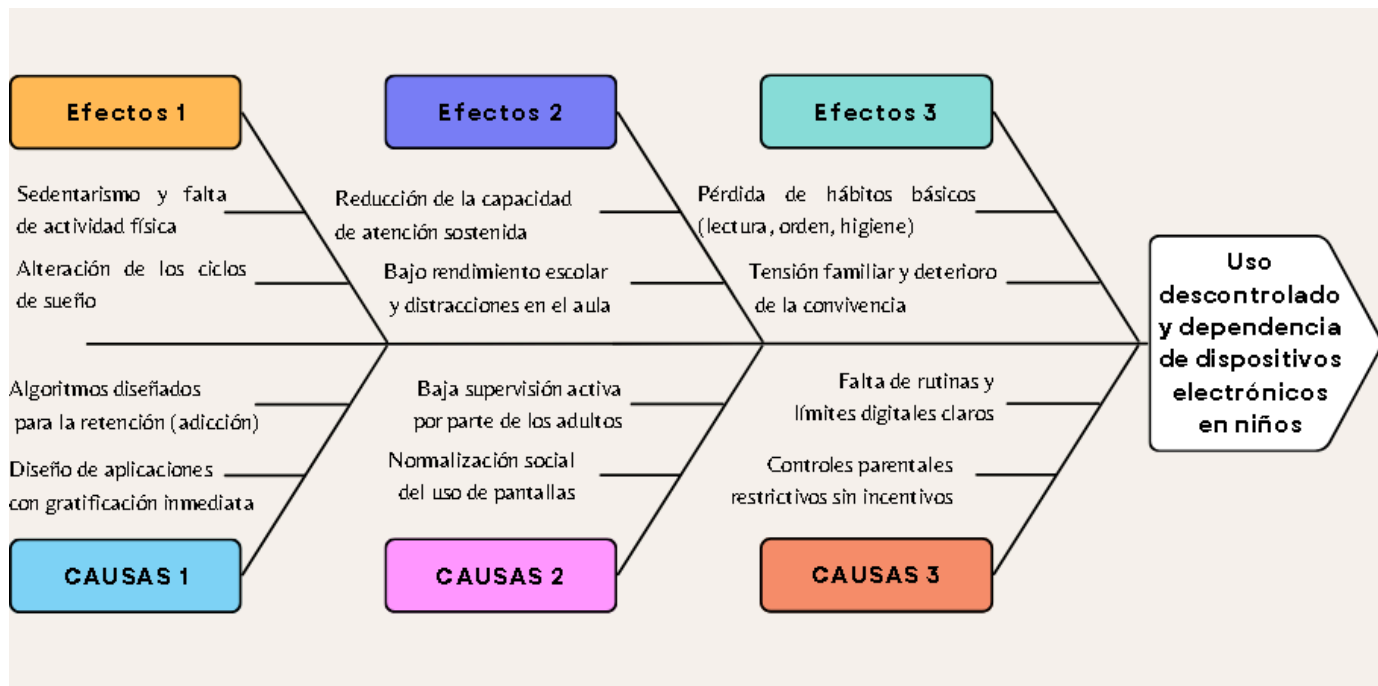
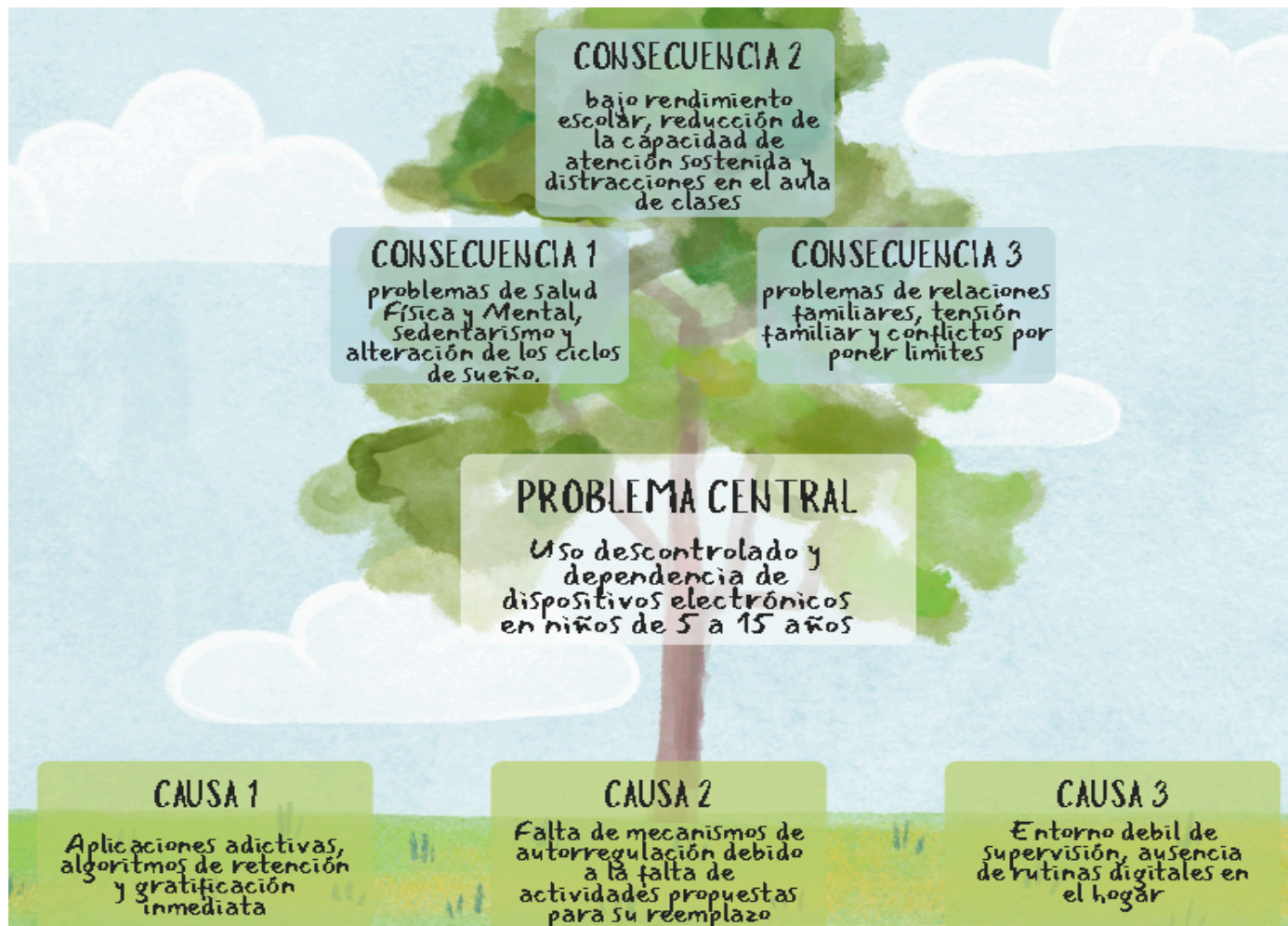


Diagrama de Ishikawa

El diagrama resume el corazón del problema que queremos resolver con MapacheSecure. No se trata solo de niños usando pantallas, sino de un círculo vicioso: las aplicaciones están diseñadas para atrapar su atención, y al no haber rutinas claras o una supervisión constante, esa distracción se convierte rápidamente en una dependencia.

Esto pega fuerte en tres áreas: la salud (por el sedentarismo y el mal sueño), el colegio (porque pierden la capacidad de concentrarse) y la familia (por las peleas al intentar poner límites).

Aquí es donde vemos nuestra oportunidad: el problema no es solo que falta control, sino que falta motivación. En lugar de solo prohibir, el análisis nos muestra que necesitamos transformar ese tiempo perdido en una experiencia positiva de autocontrol y premios.



Árbol del problema

El Árbol de Problemas revela que la dependencia infantil no es un evento aislado, sino un ciclo de retroalimentación negativa. Este ecosistema de conflicto se sostiene sobre tres pilares críticos:

Origen Técnico (Raíces): Aplicaciones diseñadas con algoritmos de retención y gratificación inmediata que anulan la capacidad de autorregulación del menor.

Impacto Sistémico (Copa): La carencia de límites técnicos deriva en sedentarismo, dispersión cognitiva en el aula y un constante desgaste de la autoridad parental.

Oportunidad de Intervención (Tronco): El problema central no es la falta de prohibición, sino la falta de motivación. Actualmente, el bloqueo es un "muro ciego" que genera fricción familiar.

MapacheSecure rompe este ciclo sustituyendo la restricción rígida por un intercambio de valor. Transformamos el tiempo de pantalla en una recompensa obtenida mediante desafíos, convirtiendo el límite en un logro personal en lugar de un castigo.

4. Descripción del mercado objetivo

4.1. Definición de los Usuarios y Clientes

MapacheSecure opera en un entorno familiar donde coexisten dos figuras clave con motivaciones distintas pero unidas por un problema común: el uso desmedido de la tecnología.

1. Usuario Final (Niños y Adolescentes): Menores de entre 5 y 15 años que han crecido en un entorno digital nativo. Para ellos, el dispositivo móvil es su principal ventana al ocio, la socialización y la información. Sin embargo, carecen de la madurez necesaria para gestionar el tiempo de uso, lo que genera frustración inmediata cuando se les impone un límite externo.
2. Cliente Decisor (Padres y Tutores): Adultos de entre 28 y 50 años, generalmente con jornadas laborales intensas, que valoran la tecnología pero se sienten sobrepasados por la gestión del tiempo de pantalla de sus hijos. Buscan una solución que no los obligue a ser el "policía" constante de la casa.

4.2. Necesidades y Dolores del Mercado

El mercado actual demanda una transición desde el control punitivo hacia la mediación tecnológica.

- Necesidades: El niño necesita estímulos que no sean frustrantes y que validen su esfuerzo. El padre necesita "paz mental", una herramienta automatizada que aplique reglas justas y preestablecidas sin necesidad de una confrontación verbal diaria.
- Dolores: La fricción familiar constante, el sentimiento de culpa en los padres por "solo prohibir", y el aburrimiento reactivo en los niños (no saber qué hacer una vez bloqueada la pantalla).

4.3. Escenario de Uso y Perfiles de Usuario

El escenario principal es el hogar, generalmente en ese bloque crítico que va desde que llegan del colegio hasta la hora de dormir. Es el momento donde los padres están cocinando o descansando y los niños buscan el refugio de las pantallas.

MapacheSecure aparece justo en ese momento de fricción. En lugar de que el padre tenga que ir a quitar el dispositivo físicamente, la aplicación toma el relevo de forma automática. El niño se encuentra con un desafío que lo saca del "trance" del video o el juego y lo invita a pensar. Si lo logra, siente la satisfacción de haber cumplido un objetivo y desbloquea sus recompensas (como puntos o más minutos), transformando un momento que solía ser de pelea en un momento de logro personal.



Mateo
10 años · Usuario principal

Hijo

INTERESES

- YouTube
- Retos
- Videojuegos

COMPORTAMIENTO

Podría pasar horas viendo contenido sin límite. No percibe el tiempo al estar en pantalla.

NECESIDAD PRINCIPAL

Que el bloqueo no se sienta como un muro, sino como un nivel de juego que puede superar.

PUNTO DE DOLOR

Los conflictos diarios con su mamá por el uso del celular le generan estrés y tensión.

LO QUE MÁS QUIERE

"Que mi mamá no se enoje conmigo todos los días por lo mismo."

Motivación

Frustración

Autonomía digital

Perfil de usuario 1



Elena
36 años · Usuario secundario

Mamá

CONTEXTO

Trabaja mucho y llega a casa buscando tranquilidad. Poco tiempo para gestionar conflictos digitales.

HERRAMIENTA CLAVE

- Control parental
- Límites de tiempo
- Retos educativos

NECESIDAD PRINCIPAL

Una herramienta que le permita delegar la decisión del tiempo de pantalla sin discutir cada día.

PUNTO DE DOLOR

Los conflictos diarios por el celular rompen la armonía familiar justo cuando más necesita descansar.

LO QUE MÁS QUIERE

"Hijo, ya sabes que MapacheSecure decide el tiempo de hoy." — y que eso sea suficiente.

META DE FONDO

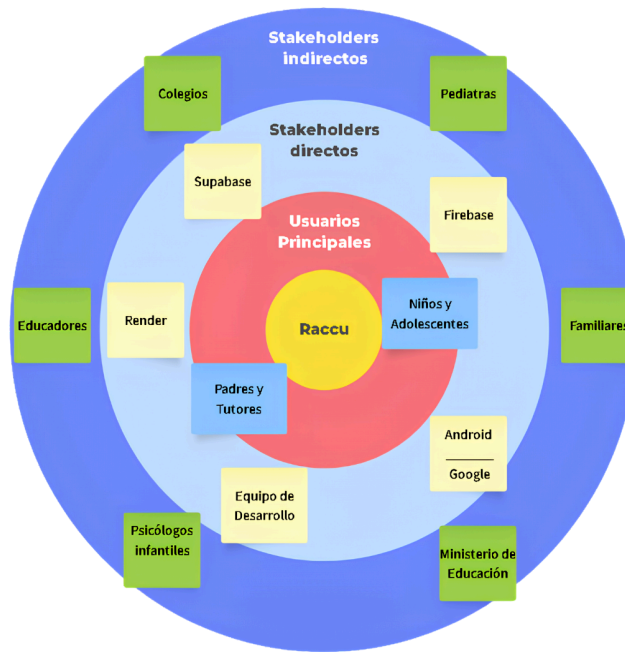
Saber que si su hijo usa el celular, al menos está pasando por retos que le aportan algo. Recuperar la armonía en la mesa.

Estrés diario

Confianza en el tech

Motivación de control

Perfil de usuario 2



Mapa de StakeHolders

Stakeholders Internos

Corresponden a los actores principales que interactúan directamente con el sistema Raccu y participan activamente en el funcionamiento de la aplicación. Este grupo está compuesto por los padres/tutores y los niños o adolescentes, quienes utilizan diariamente la plataforma para gestionar el control parental, los desafíos y el sistema de recompensas.

Stakeholders Directos

Son actores técnicos y operativos que permiten el funcionamiento del sistema y participan directamente en su desarrollo e infraestructura tecnológica. Incluye al equipo de desarrollo y a los servicios tecnológicos utilizados por la aplicación, como Android/Google, Firebase, Supabase y Render.

Stakeholders Indirectos

Corresponden a actores externos que no interactúan directamente con la aplicación, pero que influyen en el contexto educativo, social y de bienestar asociado al uso responsable de dispositivos digitales. En este grupo se encuentran colegios, educadores, psicólogos infantiles, pediatras y organismos relacionados con la educación.

5. Objetivo general y objetivos específicos

5.1. Objetivo General

Desarrollar una aplicación móvil para la plataforma Android que permita la autorregulación digital de niños y adolescentes entre 5 y 15 años, mediante un sistema de bloqueo inteligente de aplicaciones de alto consumo de entretenimiento, integrando desafíos interactivos y una economía de fichas (recompensas personalizables) para fomentar hábitos digitales saludables y reducir la dependencia a las pantallas.

5.2. Objetivos Específicos

Para lograr el objetivo general, el equipo se propone cumplir con los siguientes hitos técnicos y funcionales:

- Implementación del Módulo de Intercepción: Desarrollar un servicio en segundo plano que utilice la *Usage Stats API* de Android para detectar aplicaciones en primer plano y activar la pantalla de desafío de forma automática y funcional antes de la semana 8.
- Desarrollo del Motor de Desafíos: Crear un sistema que integre al menos tres categorías de actividades (cognitiva, física y de orden/hogar), con un mínimo de dos actividades funcionales por categoría, disponible para el usuario final antes de la semana 10.
- Gestión de Panel Parental y Autenticación: Diseñar e integrar un panel de control para el tutor que permita la configuración de horarios, selección de retos y definición de premios, protegido mediante *Supabase Auth*, operativo antes de la semana 10.
- Infraestructura de Backend y Cloud: Construir y desplegar una API funcional utilizando FastAPI y Supabase (PostgreSQL) para la persistencia de perfiles, logs de uso y sincronización de puntos, antes de la semana 9.
- Sistema de Notificaciones Push: Integrar *Firebase Cloud Messaging* (FCM) para el envío de alertas de logros desbloqueados al menor y resúmenes diarios de actividad al padre, verificado en dispositivos físicos antes de la semana 11.

6. Situación inicial del cliente (AS-IS)

6.1. Descripción del Proceso Actual

En la actualidad, la gestión del tiempo de pantalla en el hogar es un proceso 100% manual, reactivo y generador de conflictos. El ciclo habitual comienza cuando el niño accede al dispositivo móvil sin restricciones técnicas previas. Al no existir un sistema de alerta o bloqueo automático, el menor pierde la noción del tiempo debido al diseño de gratificación inmediata de las aplicaciones de entretenimiento.

El control depende exclusivamente de la vigilancia visual del adulto. El proceso se rompe en los siguientes puntos críticos:

- **Detección Tardía:** El padre suele notar el exceso de uso cuando ya han pasado varias horas, lo que genera una reacción de culpa en el adulto y frustración en el niño.
- **Intervención Física:** La restricción ocurre mediante la retirada física del dispositivo o el apagado abrupto, lo que se percibe como una agresión arbitraria.
- **Negociación Incesante:** Se inicia un ciclo de "cinco minutos más" que desgasta la autoridad parental y no educa en la autorregulación.
- **Falta de Registro:** No existe un dato real de cuánto tiempo se consumió ni en qué aplicaciones, imposibilitando la toma de decisiones informadas.

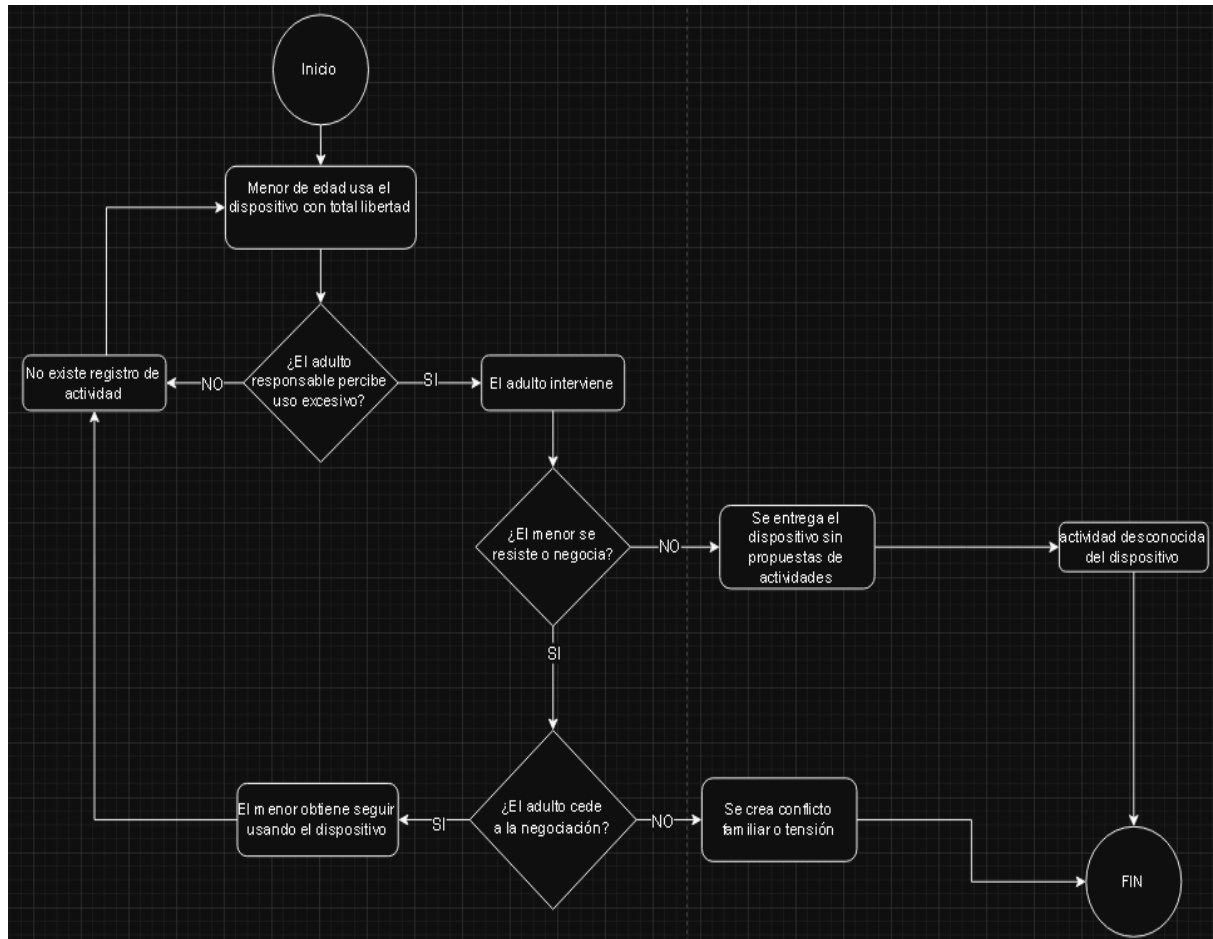
6.2. Puntos de Dolor Identificados

Este proceso "analógico" genera cuatro problemas graves:

1. **Fricción Familiar:** El clima del hogar se tensa diariamente por la "guerra de las pantallas".
2. **Inexistencia de Alternativas:** Al quitar el celular, el niño queda en un vacío de actividad, lo que dispara la irritabilidad.
3. **Carencia de Incentivos:** La única herramienta es la prohibición; no hay un premio por dejar el teléfono voluntariamente.
4. **Dependencia del Adulto:** El sistema falla si el padre está ocupado, trabajando o descansando.

6.3. Diagrama de Flujo del Proceso Actual (AS-IS)

A continuación, se representa gráficamente la secuencia de eventos y las decisiones que hoy en día terminan en conflicto familiar.



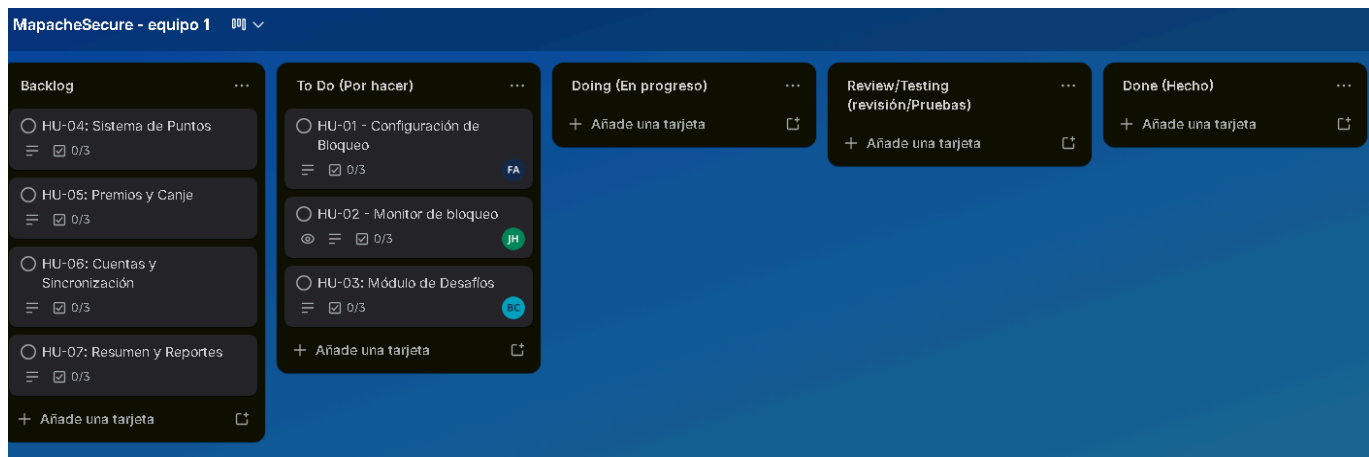
Flujo de interacción manual y puntos de fricción en el control parental tradicional.

7. Planificación

7.1. Metodología de Gestión: SCRUM y Trello

Para asegurar el cumplimiento de los hitos en el plazo establecido, el equipo ha implementado un marco de trabajo Ágil (Scrum). La gestión visual y el control de flujos se centralizan en un tablero de Trello, el cual permite organizar el trabajo mediante Historias de Usuario (HU) distribuidas en columnas de estado: *Backlog*, *To Do* (Por hacer), *Doing* (En progreso), *Review/Testing* (Revisión) y *Done* (Hecho).

Como se observa en la Imagen, el equipo ha desglosado las funcionalidades críticas (HU-01 a HU-07). Actualmente, el foco está en la configuración del monitor de bloqueo y el módulo de desafíos, permitiendo una trazabilidad total del avance y una rápida detección de cuellos de botella técnicos.



<https://trello.com/invite/b/69ca8b5f9fe047ba0b854111/ATTI62d76a73cad5160a7aaff5eeef843617690313D6/mapachesequipo-1>

7.2. Organización del Equipo y Roles Críticos

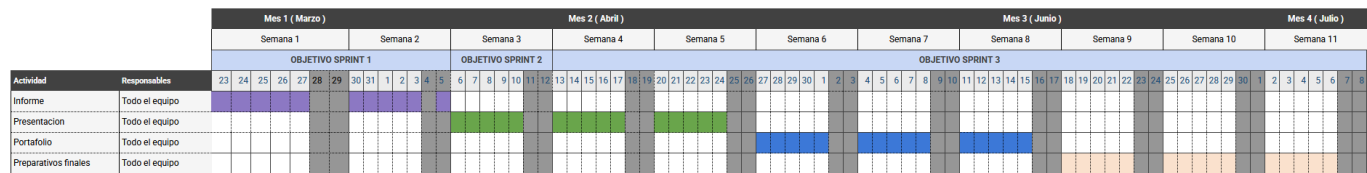
La distribución de responsabilidades asegura que cada capa de la aplicación (Mobile, API y Datos) tenga un responsable directo, aunque la ejecución de los entregables principales es colaborativa:

- Benjamín Castillo (Líder de Proyecto, DBA y Analista Funcional): Responsable de la arquitectura de datos en Supabase, el diseño de la lógica de negocio y la administración del tablero Trello.
- Javier Vieytes (Desarrollador Backend y Especialista Técnico): Encargado de la construcción de la API en FastAPI, la integración con Firebase y el despliegue en la plataforma Render.
- Fernanda Arraño (QA, Testing y Desarrolladora Frontend Mobile): Responsable de la interfaz de usuario en Android, la lógica de intercepción de aplicaciones y el aseguramiento de la calidad.

7.3. Cronograma de Actividades (Carta Gantt)

El proyecto se desarrolla a lo largo de 11 semanas, organizadas en tres Sprints principales de trabajo. A continuación, se presenta la representación gráfica de la planificación temporal para los entregables clave del semestre:

MapacheSecure



<https://docs.google.com/spreadsheets/d/1Re5Rkkl2143U-21JFFkwfK4Rm3SaJ8x169OaGsRKOZk/edit?gid=207134840#gid=207134840>

8. Servicios Cloud a utilizar (IaaS / PaaS / SaaS)

Para garantizar la disponibilidad y escalabilidad de **MapacheSecure**, se ha seleccionado un stack tecnológico basado en servicios en la nube que permiten un despliegue ágil y sin costos de infraestructura inicial:

- **Render (PaaS):** Se utiliza para el hosting del backend desarrollado en FastAPI. Render permite la integración continua con GitHub, desplegando automáticamente cada cambio en el código y proporcionando certificados SSL (HTTPS) de forma nativa.
- **Supabase (DBaaS/PaaS):** Actúa como nuestra base de datos. Además de almacenar perfiles y puntos, se utiliza su servicio de autenticación para proteger el panel de control parental mediante tokens seguros.
- **Firebase Cloud Messaging (SaaS):** Es el servicio encargado de la mensajería push. Permite enviar alertas en tiempo real desde el backend hacia el dispositivo Android del niño (logros) y del padre (reportes de uso).

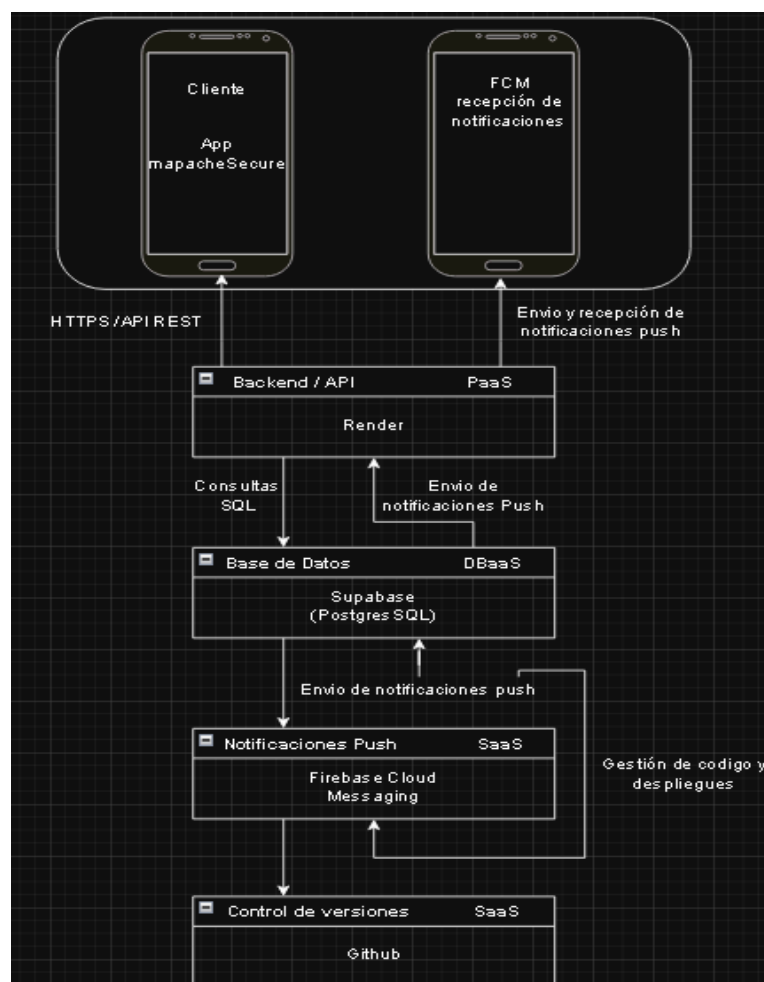


Diagrama de arquitectura servicio cloud

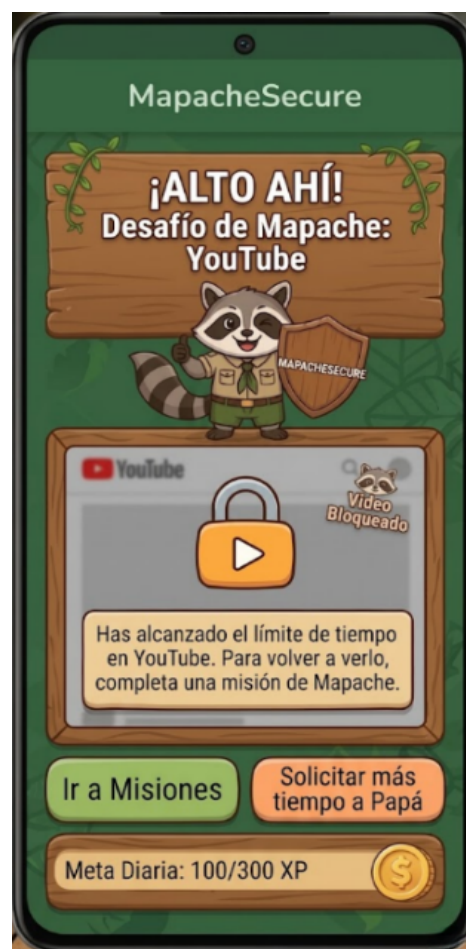
9. Conceptualización (Solución propuesta – Versión preliminar)

Con **MapacheSecure** instalado, el proceso de uso de dispositivos cambia completamente. Cuando el niño intenta abrir una aplicación de entretenimiento bloqueada, el sistema intercepta ese intento en tiempo real y muestra una pantalla de desafío en su lugar. El niño debe completar una actividad para poder acceder. Al terminarla, gana puntos que acumula en su perfil. Esos puntos los puede canjear por las recompensas que su padre configuró previamente.

Funcionalidades Principales:

1. Módulo de Intercepción y Bloqueo Inteligente:

El sistema utiliza un servicio en segundo plano que monitorea el "Usage Stats API" de Android. Cuando detecta que una aplicación está marcada como "entretenimiento" (ej. TikTok, YouTube, Roblox) pasa a primer plano, la app superpone inmediatamente una pantalla de bloqueo personalizada.



Bloqueo inteligente

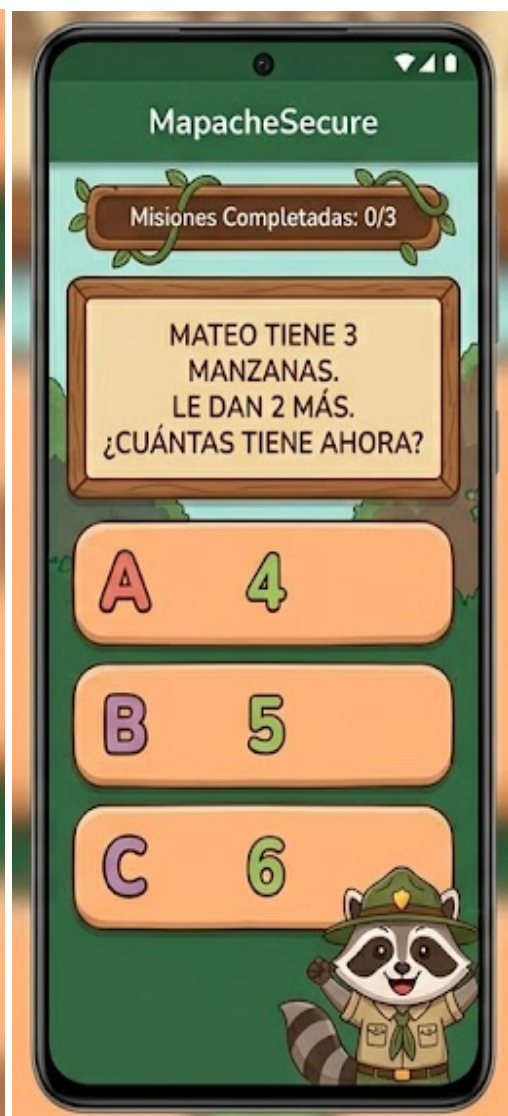
2. Motor de Desafíos Multimodal:

Es el núcleo pedagógico de la app. No se limita a un solo tipo de tarea, sino que ofrece tres categorías para evitar la fatiga y fomentar el desarrollo integral:

- **Cognitivos:** Micro-lecturas con comprensión, trivias o rompecabezas lógicos.
- **Físicos:** Retos detectados mediante sensores (acelerómetro) como realizar 20 saltos.
- **De Orden/Hogar:** Tareas como hacer la cama, enviando una foto como evidencia.



Motor de desafíos



Desafío cognitivo



Evidencia del desafío completado

3. Sistema de Economía de Fichas (Puntos y Recompensas):

Basado en psicología conductual, cada desafío superado otorga "MapachePoints". El sistema incluye un catálogo de recompensas configurables donde el niño canjea sus puntos por beneficios (ej. "30 minutos extra de juego" o "elegir la película").



Tienda de recompensas

4. Panel de Control Parental (Gestor de Reglas):

Interfaz exclusiva para el adulto, protegida por autenticación biométrica o PIN. Permite:

- Establecer "Horas de Enfoque" (bloqueo automático).
- Seleccionar qué aplicaciones específicas requieren desafíos para abrirse.
- Personalizar la dificultad de los retos según la edad del menor.



Panel de control parental

5. Sistema de Notificaciones y Reportes:

Mediante **Firestore Cloud Messaging (FCM)**, el sistema mantiene un flujo de comunicación constante:

- **Para el niño:** Alertas de "logro desbloqueado" o recordatorios de desafíos pendientes.
- **Para el padre:** Notificaciones sobre el cumplimiento de metas y un reporte diario resumido que indica el tiempo total de pantalla y el nivel de cumplimiento de los desafíos propuestos.



Panel del niño

Panel del padre

6. Arquitectura de Sincronización Cloud:

Utiliza un backend desarrollado en FastAPI que actúa como puente entre el dispositivo del niño y el del padre. Toda la progresión, puntos acumulados y logs de intentos de apertura se sincronizan en tiempo real con una base de datos PostgreSQL en Supabase. Esto garantiza que, aunque se cambie de dispositivo, el historial y los puntos se mantengan íntegros.

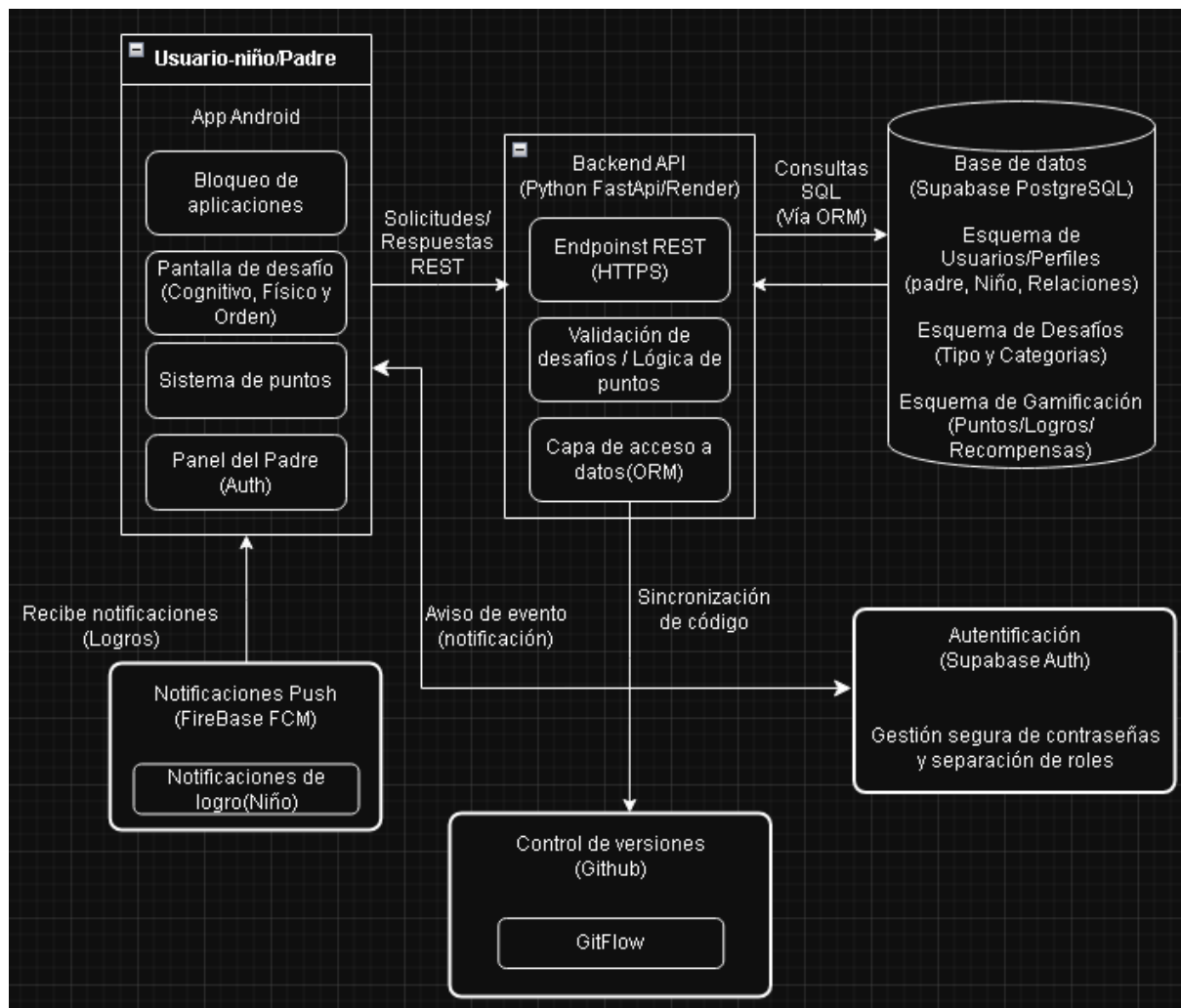


Diagrama de arquitectura proceso de aplicación

10. Alcance, supuestos y restricciones

Alcance del Proyecto

MapacheSecure contempla el desarrollo de una aplicación móvil Android orientada al control parental y la autorregulación digital infantil mediante un enfoque gamificado. El sistema incluirá un mecanismo de bloqueo inteligente de aplicaciones de entretenimiento utilizando la UsageStats API de Android, permitiendo detectar aplicaciones en primer plano e interceptar su uso mediante desafíos interactivos antes de autorizar el acceso.

La solución incorpora un motor de desafíos compuesto por actividades cognitivas, físicas y de orden/hábitos del hogar, diseñadas para reemplazar el tiempo de pantalla por actividades saludables y educativas. Cada desafío completado otorgará puntos acumulables dentro de un sistema de economía de fichas, los cuales podrán ser canjeados por recompensas configuradas previamente por el padre o tutor.

El sistema considera además:

- Panel parental protegido mediante autenticación segura con JWT.
- Gestión de aplicaciones bloqueadas por horarios.
- Sistema de gamificación con avatar evolutivo "Raccu".
- Personalización de foto de perfil del hijo.
- Generación de desafíos con IA.
- Backend REST desarrollado con FastAPI.
- Persistencia de datos mediante Supabase/PostgreSQL.
- Notificaciones push utilizando Firebase Cloud Messaging.
- Emails transaccionales mediante Resend.
- Despliegue cloud en Render.
- Landing page informativa publicada en raccu.cl.

El alcance contempla un prototipo funcional orientado a validación técnica y académica, no una solución comercial final lista para producción masiva.

Exclusiones del Proyecto

El proyecto no contempla el desarrollo de:

- Versión para iOS o multiplataforma fuera de Android.
- Compatibilidad con dispositivos Android inferiores a la versión 8.0.
- Compatibilidad específica para tablets o Smart TVs.
- Módulo de pagos online o monetización.
- Marketplace de recompensas digitales automatizadas.
- Plataforma web independiente para el panel parental.

- Soporte multilenguaje.
- Sincronización offline o funcionamiento sin conexión a internet.
- Integración con servicios externos de control parental de terceros.

La funcionalidad de generación dinámica de desafíos mediante IA queda únicamente como propuesta futura asociada al Sprint 3.

Entregables

Los entregables oficiales del proyecto corresponden a:

- Informe técnico completo del sistema.
- Repositorio GitHub del frontend y backend.
- Aplicación Android instalable (.apk).
- Backend desplegado y funcional en Render.
- Base de datos operativa en Supabase.
- Landing page publicada en raccu.cl.
- Diagramas técnicos y documentación de arquitectura.
- Evidencias funcionales y pruebas del sistema.
- Configuración básica de entorno cloud y variables de entorno.

Supuestos

Para el correcto funcionamiento y desarrollo del proyecto se asume que:

- Los dispositivos de prueba utilizan Android 8.0 o superior.
- El equipo dispone de acceso estable a internet durante el desarrollo.
- Los servicios gratuitos de Render y Supabase mantendrán disponibilidad durante el semestre.
- El usuario padre posee conocimientos básicos para configurar la aplicación.
- El dispositivo del hijo mantiene habilitados los permisos necesarios del sistema (Usage Access, superposición, notificaciones).
- Los niños no tienen acceso a las credenciales administrativas del padre.
- Firebase FCM mantendrá compatibilidad con la configuración Android utilizada.
- La cantidad de usuarios durante la etapa académica será reducida y compatible con los límites gratuitos de infraestructura cloud.

Restricciones

El proyecto presenta las siguientes restricciones técnicas y operativas:

- Duración máxima de 11 semanas organizadas en tres sprints Scrum.
- Equipo reducido de 3 integrantes sin dedicación exclusiva.

- Ausencia de presupuesto para infraestructura o APIs de pago.
- Dependencia directa de servicios cloud externos gratuitos.
- Limitaciones impuestas por Android respecto al monitoreo de aplicaciones en segundo plano.
- Restricciones de consumo y suspensión automática del plan gratuito de Render tras períodos de inactividad.
- Restricciones de permisos del sistema Android que pueden variar según fabricante del dispositivo.
- Alcance académico limitado a un MVP funcional y no a un producto comercial completo.
- Capacidad limitada de pruebas masivas en múltiples dispositivos reales.

11. Metodología de desarrollo y gestión del proyecto

Metodología Scrum

Para el desarrollo de MapacheSecure se adoptó la metodología ágil Scrum, complementada con el uso de Trello como herramienta de gestión visual del flujo de trabajo.

Scrum resulta adecuada para este proyecto por varias razones. En primer lugar, el equipo es pequeño (3 integrantes), lo que se alinea perfectamente con la estructura ligera que propone esta metodología. En segundo lugar, el proyecto tiene un plazo definido de 11 semanas dividido en sprints, lo que permite entregar avances concretos y funcionales en cada iteración en lugar de esperar al final para tener resultados visibles.

Cada sprint tiene un objetivo claro: el primero se enfocó en la planificación y diseño del sistema, el segundo en el desarrollo del núcleo técnico (backend, base de datos y módulo de bloqueo), y el tercero en la integración, pruebas y preparación del entregable final. Esta división permite al equipo detectar problemas técnicos de forma temprana y ajustar el desarrollo sin afectar el plazo total.

El tablero Trello organiza el trabajo mediante historias de usuario (HU-01 a HU-07) distribuidas en columnas de estado: Backlog, To Do, Doing, Review y Done, lo que da visibilidad total del avance a todos los integrantes en tiempo real.

A diferencia de metodologías tradicionales como el modelo en cascada, Scrum permite incorporar cambios durante el desarrollo, algo especialmente relevante en un proyecto universitario donde los requerimientos pueden ajustarse según el feedback del docente o las limitaciones técnicas que surjan en el camino.

Planificación, Seguimiento y Control Semanal

Nuestro equipo se organizará en ciclos semanales enmarcados dentro de tres sprints principales que abarcan las 11 semanas del semestre. Cada sprint tiene un objetivo concreto y medible que guía las decisiones del equipo durante ese período.

Planificación

Al inicio de cada sprint el equipo realiza una reunión de planificación donde se revisan las historias de usuario pendientes en el backlog de Trello, se estima el esfuerzo requerido para cada tarea y se asignan responsabilidades según los roles definidos: Benjamín en arquitectura de datos y lógica de negocio, Javier en backend y despliegue, y Fernanda en frontend móvil y control de calidad. Las tareas seleccionadas se mueven a la columna "To Do" del tablero, estableciendo así el compromiso de trabajo para la semana.

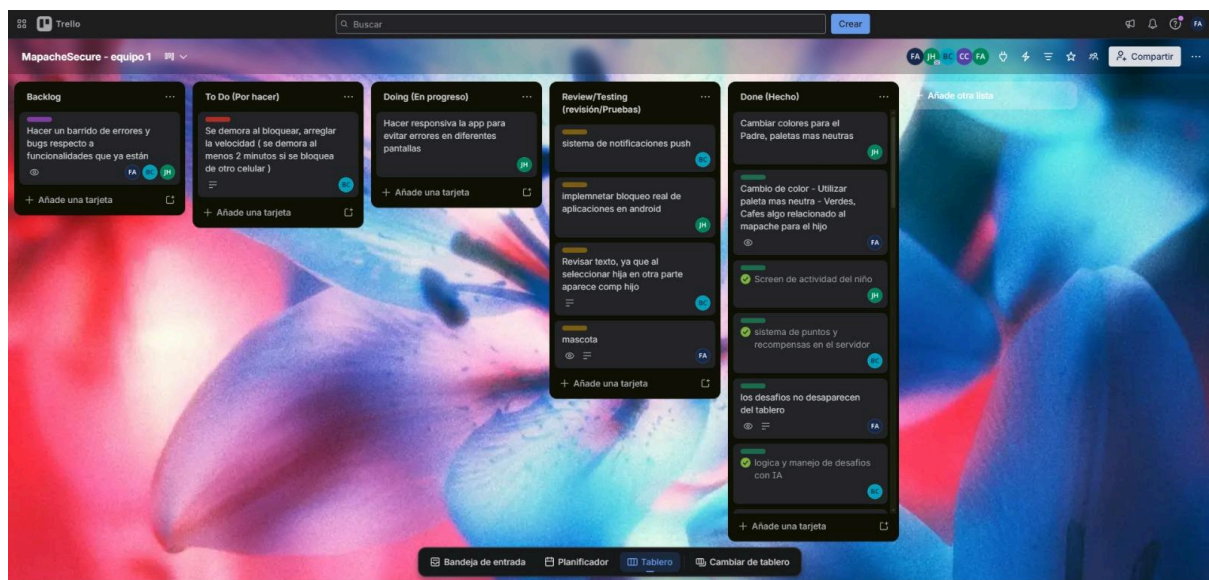
Seguimiento

Durante la semana cada integrante actualiza el estado de sus tareas en el tablero Trello, moviendo las tarjetas entre las columnas "Doing", "Review" y "Done" según el avance real.

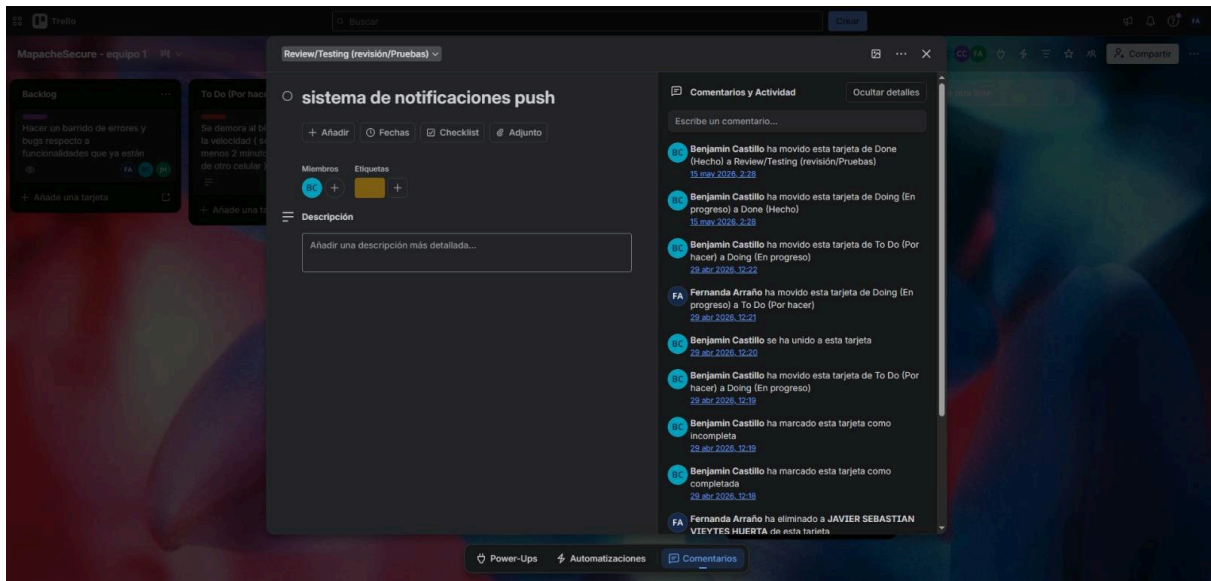
El repositorio de GitHub actúa como registro técnico del progreso, ya que cada commit representa un avance concreto y verificable. El equipo se comunica de forma asíncrona durante la semana mediante WhatsApp para resolver bloqueos técnicos o coordinar decisiones que afecten al trabajo de otro integrante.

Control

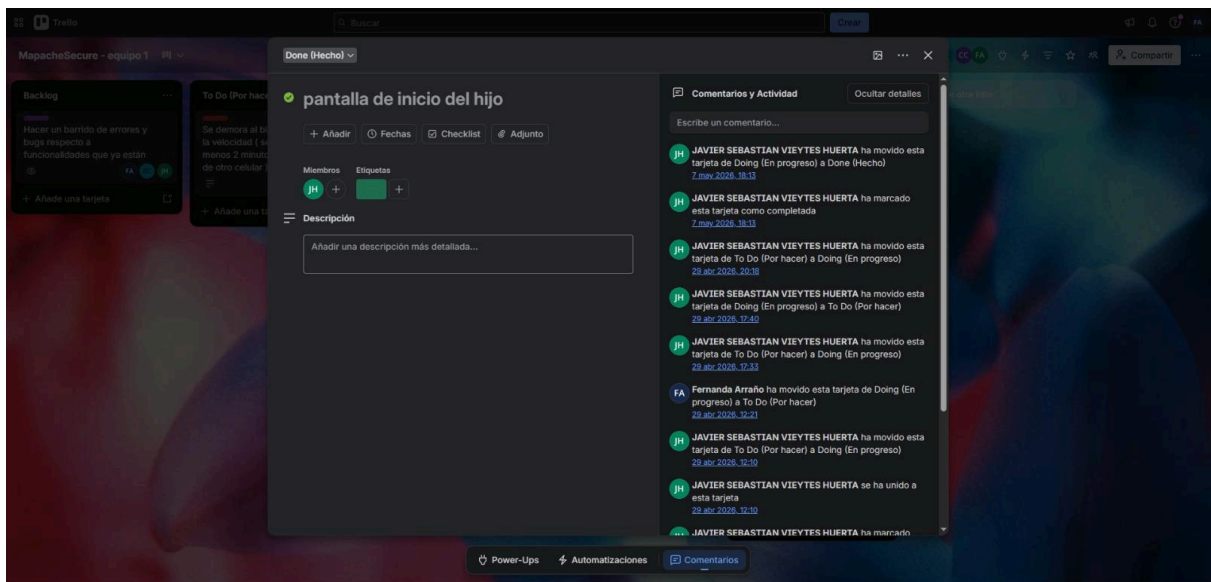
Al final de cada semana el equipo realiza una revisión informal donde se evalúa qué tareas se completaron, qué quedó pendiente y por qué. Las tareas no completadas se reincorporan al backlog con una nueva estimación. Adicionalmente, al cierre de cada sprint se realiza una retrospectiva breve donde el equipo identifica qué funcionó bien, qué debe mejorar y qué ajustes se aplicarán al siguiente sprint. Este ciclo de inspección y adaptación continua es el núcleo de la metodología Scrum y permite al equipo mantener el control del proyecto sin depender de una planificación rígida que no contemple los imprevistos propios del desarrollo de software.



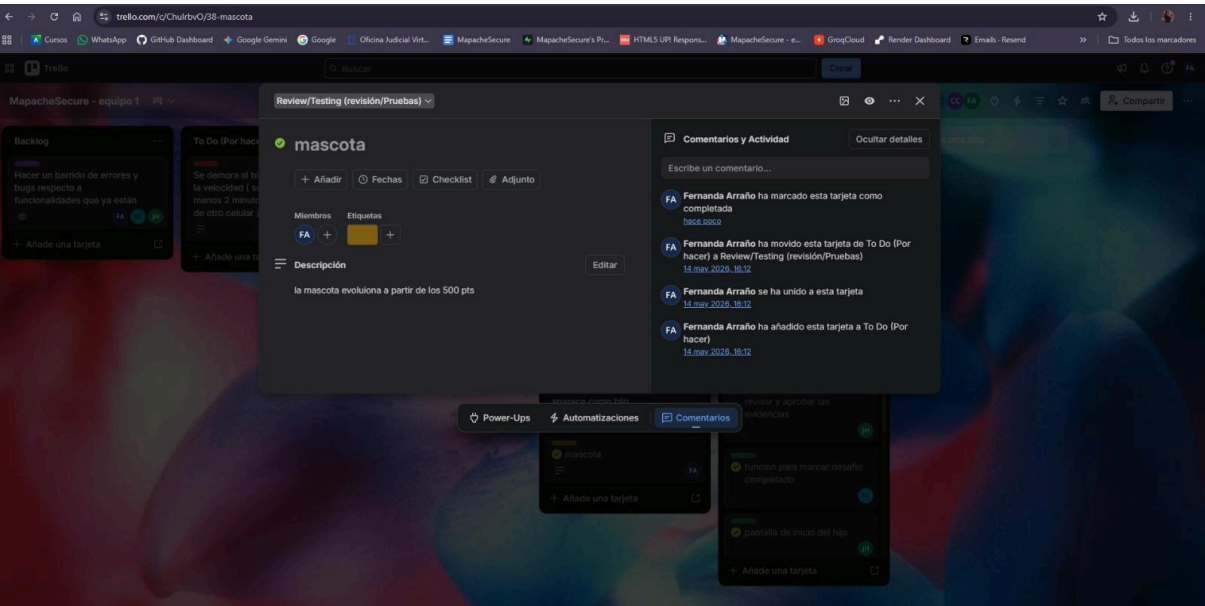
Tablero Trello 1



Tablero Trello 2



Tablero Trello 3



Tablero Trello 4

Integrante	Rol Scrum	Responsabilidades Principales	Tecnologías
Benjamín Castillo	Líder /BDA	Diseño del modelo de datos, lógica de negocio, estructura de tablas en Supabase	Supabase, Python
Javier Vieytes	Backend/cloud	Desarrollo del backend, despliegue y mantención del servidor en producción	FastAPI, Python, Render
Fernanda Arraño	Frontend/QA	Desarrollo del frontend móvil Android, control de calidad y pruebas	Flutter, Dart, Android

Roles del equipo (quién hace qué) y responsabilidades.

Definición de “Hecho” (Definition of Done)

Para MapacheSecure, un elemento del backlog se considera "hecho" cuando cumple con todos los criterios siguientes:

El código ha sido desarrollado, probado manualmente por el desarrollador responsable y revisado por al menos un integrante del equipo. La funcionalidad implementada responde correctamente a los casos de uso esperados, incluyendo tanto los flujos exitosos como los casos de error más comunes. El endpoint o módulo desarrollado ha sido probado en el entorno local mediante la documentación interactiva de la API (Swagger UI en /docs) o directamente desde la aplicación Android según corresponda. Los cambios han sido commiteados al repositorio de GitHub con un mensaje descriptivo y el código no genera errores al iniciar el servidor. Finalmente, la funcionalidad ha sido desplegada exitosamente en el entorno de producción (Render) y responde correctamente desde la URL pública.

Funcionalidad	Revisado	Probado Swagger/app	Commiteado en Github	Desplegado en render
Autenticación	✓	✓	✓	✓
Gestion de usuarios	✓	✓	✓	✓
Módulo de desafíos	✓	✓	✓	✓
Sistema de recompensas	✓	✓	✓	✓
Apps bloqueadas	✓	✓	✓	✓
Avatar y evolucion de mascota	✓	✓	✓	✓
Foto de perfil del hijo	✓	✓	✓	✓
Notificaciones FCM	✓	-	✓	En progreso

Criterios de calidad mínimos

Los criterios de calidad mínimos que debe cumplir cada entregable son los siguientes. Todos los endpoints de la API deben retornar códigos HTTP correctos (200 para éxito, 404 para recurso no encontrado, 500 para errores del servidor). Todas las respuestas de la API deben estar en formato JSON válido y seguir una estructura consistente: las respuestas exitosas deben incluir los datos solicitados junto a un mensaje descriptivo, mientras que las respuestas de error deben incluir siempre el campo **detail** con una descripción clara del problema ocurrido, permitiendo que la app Android pueda interpretarlas y mostrar mensajes apropiados al usuario.

La base de datos no debe contener registros huérfanos ni inconsistencias entre tablas relacionadas. El módulo de bloqueo en Android debe interceptar correctamente las aplicaciones marcadas como bloqueadas sin afectar el rendimiento general del dispositivo. El sistema de puntos debe reflejar con exactitud los desafíos completados y validados, sin duplicar ni perder registros. La interfaz del panel parental debe estar protegida por autenticación y no debe ser accesible sin credenciales válidas. Por último, la aplicación debe funcionar correctamente en dispositivos Android con versión 8.0 o superior, que es el mínimo definido en el alcance del proyecto.

Sprint	Periodo	Objetivo principal
1	16 mar - 5 abril	Investigación, planificación y documentación del proyecto: levantamiento de requerimientos, diseño de diagramas wireframes y arquitectura.
2	7 abril - 24 may	Desarrollo e implementación de la aplicación: backend (FastAPI), base de datos (Supabase), módulo de bloqueo(UsageStats), sistema de desafíos, recompensas y mascota.
3	26 may - 30 jun	Integración final, pruebas de sistema, correcciones y preparación del entregable definitivo: notificaciones push (FCM), módulo IA y pulido de la app

Lista de hitos o sprints con fechas

12.Documentos y diagramas de diseño de la solución

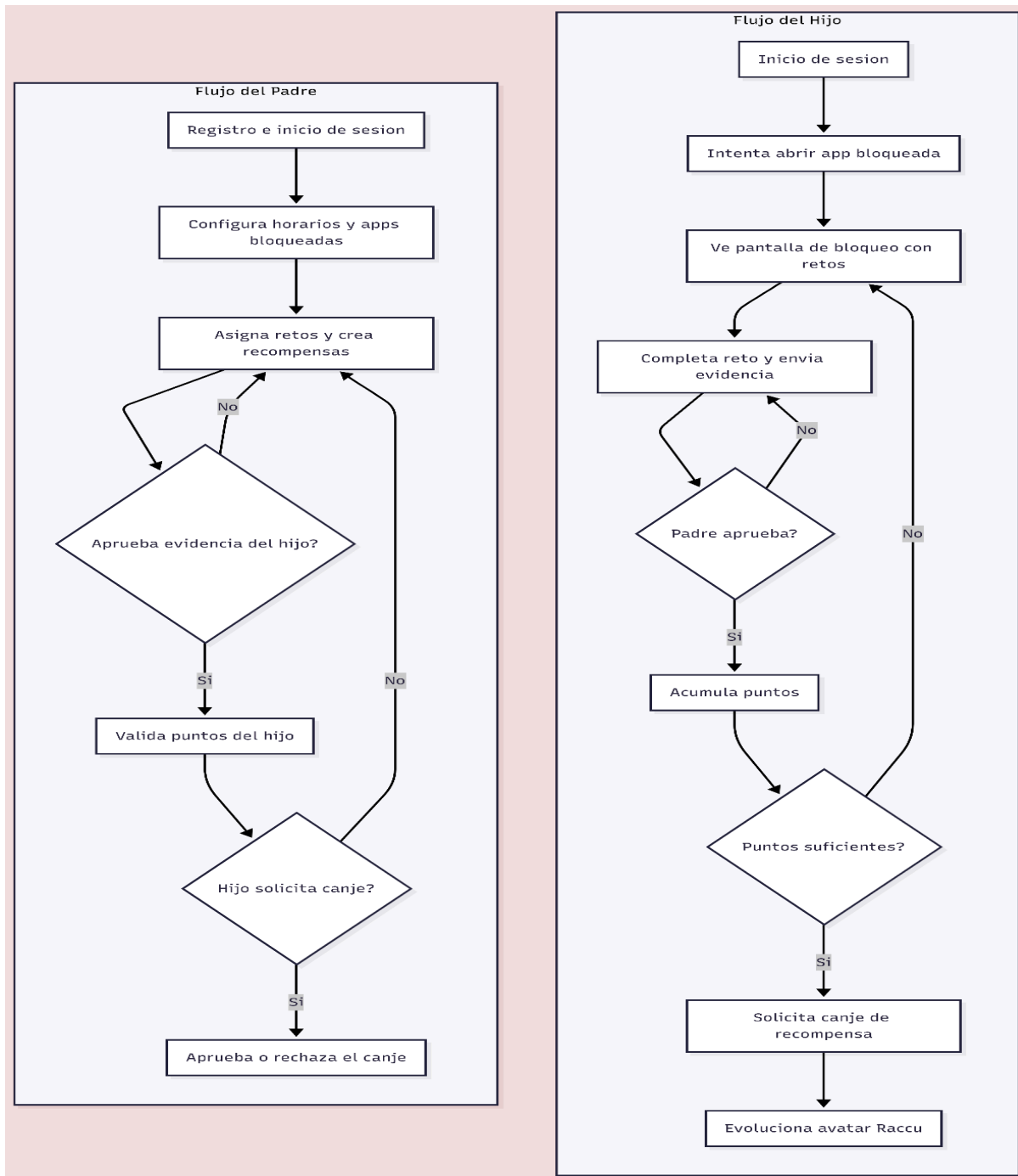


Diagrama de flujo del usuario (User Flow) o BPMN del proceso.

Propósito:

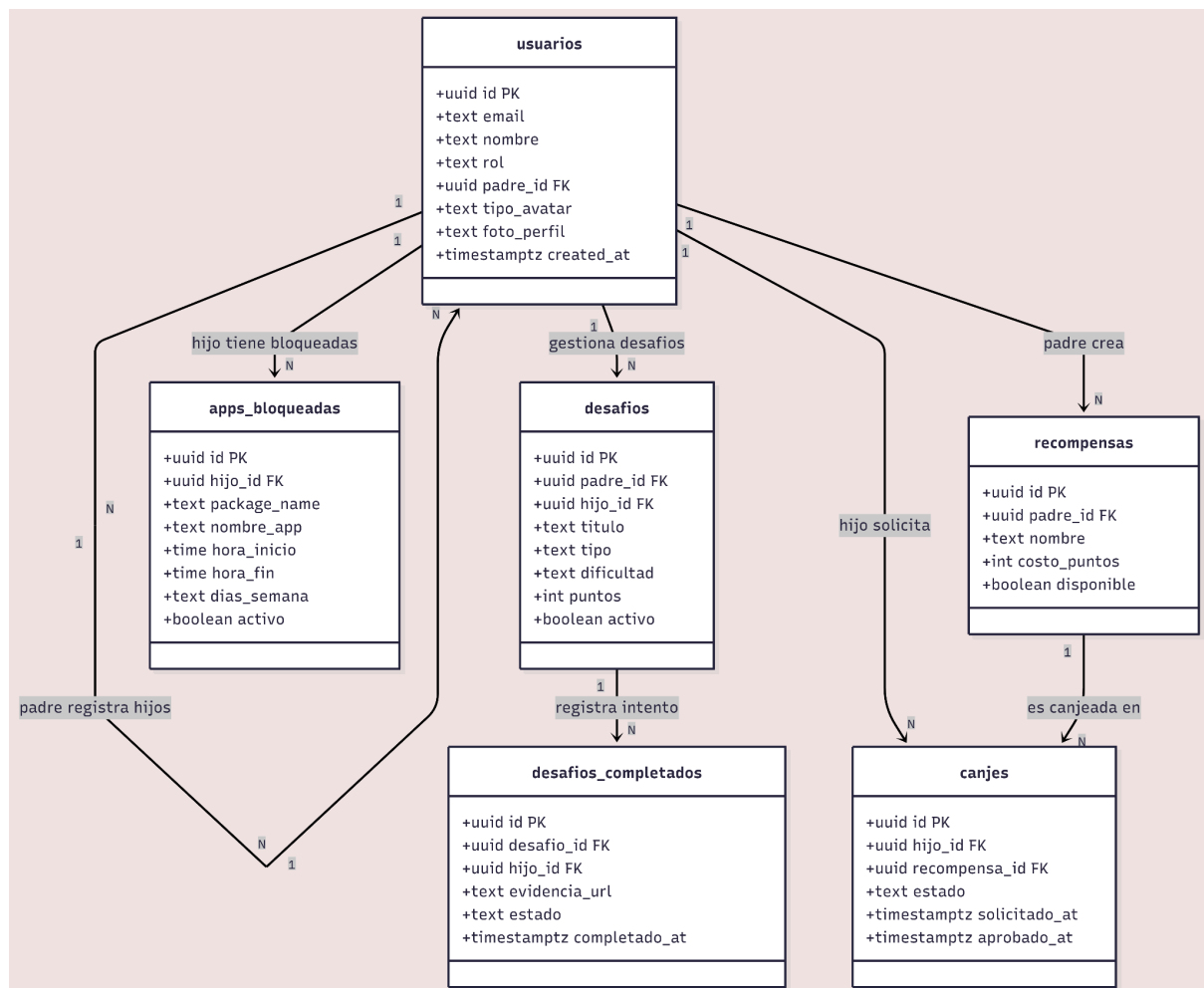
Mostrar cómo interactúan el hijo y el padre dentro del sistema.

Qué representa:

Los flujos de interacción de los dos actores principales del sistema

Flujo del padre: registra al hijo, configura bloques de horario para bloquear aplicaciones (mínimo de 2 horas), asigna retos y define mínimo 3 recompensas disponibles, aprueba o rechaza evidencias enviadas por el hijo, y aprueba las recompensas solicitadas para el próximo ciclo.

Flujo del hijo: visualiza sus retos pendientes, los completa y envía evidencia fotográfica, acumula puntos para canjear recompensas, personaliza su foto de perfil con imágenes predeterminadas de Raccu, y sube de nivel a su avatar según los puntos acumulados.



Modelo de datos (MER/DER) o esquema de BD (tablas, relaciones)

Propósito:

Definir la estructura de la base de datos del sistema.

Qué representa: La estructura completa de la base de datos del sistema, compuesta por 6 tablas.

- usuarios — almacena perfiles de padres e hijos, incluyendo configuración de bloqueo, avatar y foto de perfil
- desafios — retos asignados por el padre al hijo, con tipo, dificultad y puntos asociados

- desafios_completados — registro de cada reto completado por el hijo con su evidencia y estado de validación
- apps_bloqueadas — aplicaciones restringidas por horario para cada hijo
- recompensas — beneficios creados por el padre canjeables con puntos
- canjes — historial de recompensas solicitadas por el hijo y aprobadas por el padre

Relación con el problema:

Permite almacenar y gestionar el progreso del hijo, el control parental y el sistema de puntos.

Relación con los objetivos:

Es la base para implementar la lógica del sistema y asegurar que la información se maneje correctamente.

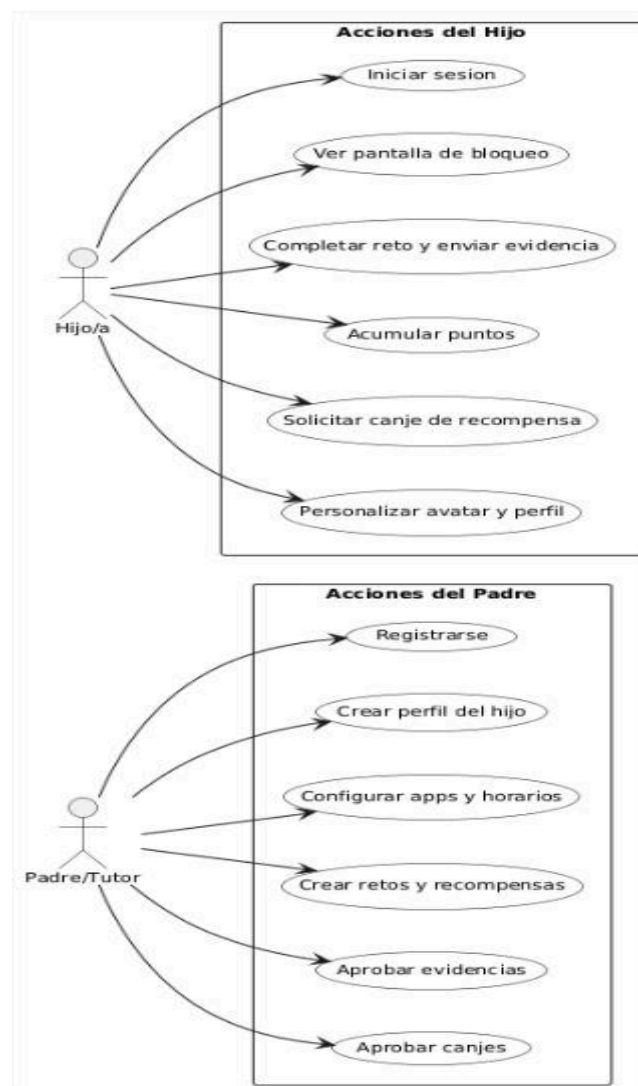


Diagrama de casos de uso o historias de usuario (con roles/actores).

Propósito:

Representar las funcionalidades del sistema desde la perspectiva de los usuarios (padre e hijo).

Qué representa:

Las funcionalidades del sistema desde la perspectiva de los dos actores principales y sus interacciones

Relación con el problema:

Permite identificar cómo cada actor interactúa con el sistema para controlar el uso de aplicaciones, gestionar desafíos y motivar al hijo mediante recompensas y progresión del avatar

Relación con los objetivos:

Asegura que todas las funcionalidades necesarias estén cubiertas: control parental, desafíos y evidencias, recompensas canjeables y gamificación mediante el avatar Raccu.

Historias de usuario.

HU-01: Como padre, quiero registrarme con mi correo y contraseña, para tener una cuenta personal y poder acceder al panel de control parental.

HU-02: Como padre, quiero crear un perfil para mi hijo dentro de la app, para vincularlo a mi cuenta y poder manejar sus horarios

HU-03: Como hijo, quiero iniciar sesión con las credenciales que ingresó mi padre, para acceder a la app desde mi dispositivo y ver los desafíos asignados con los puntos ganados.

HU-04: Como padre, quiero seleccionar qué aplicaciones deseas bloquear en el dispositivo de mi hijo, para poder limitar el acceso en horarios educativos o para que descanse de este.

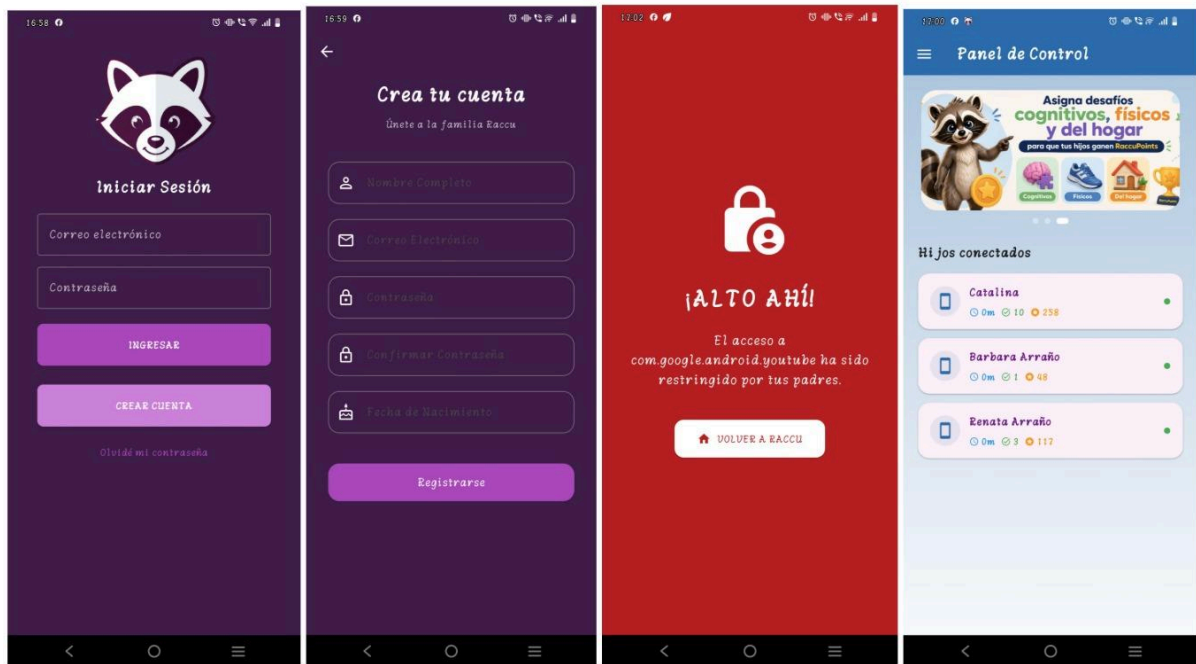
HU-05: Como hijo, quiero poder ver una pantalla de bloqueo cuando intento abrir una app restringida, para entender que es mi momento de parar de ocuparme y realizar mis tareas/desafíos.

HU-06: Como hijo, quiero algún tipo de recompensa al completar el desafío, así me da una motivación a seguir completando las

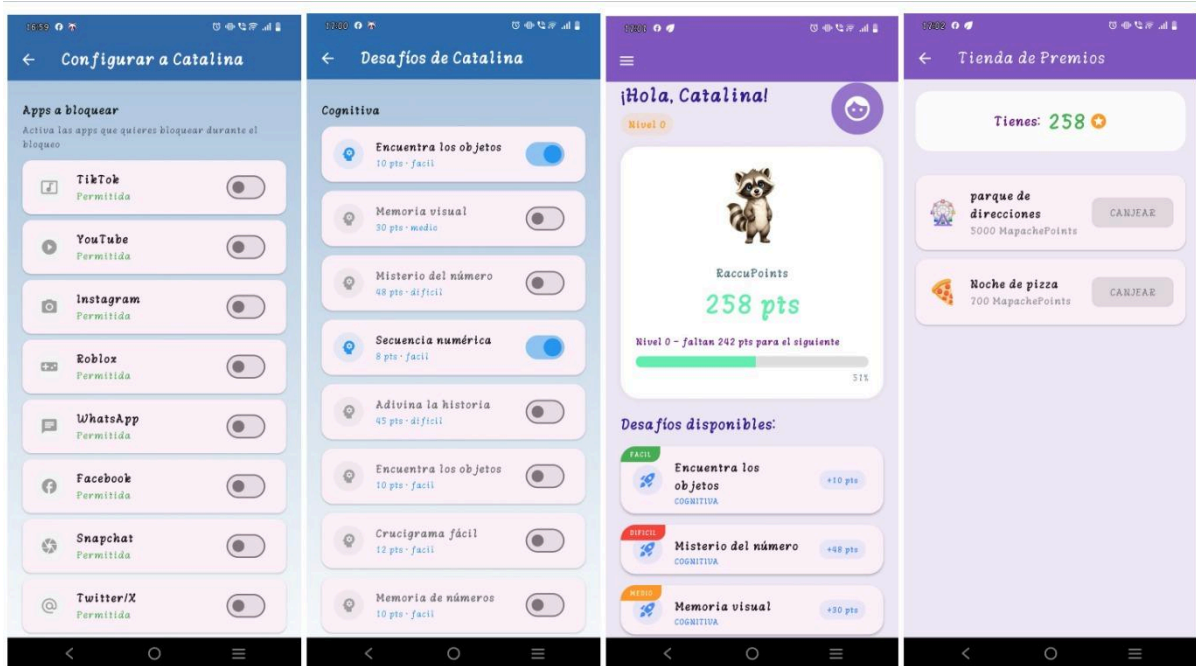
HU-07: Como hijo, me gustaría poder ver los desafíos que tengo pendientes, para saber que debo hacer y completarlos para ganar los puntos.

HU-08: Como padre, me gustaría poder crear o generar desafíos para que mi hijo no se quede sin desafíos a realizar.

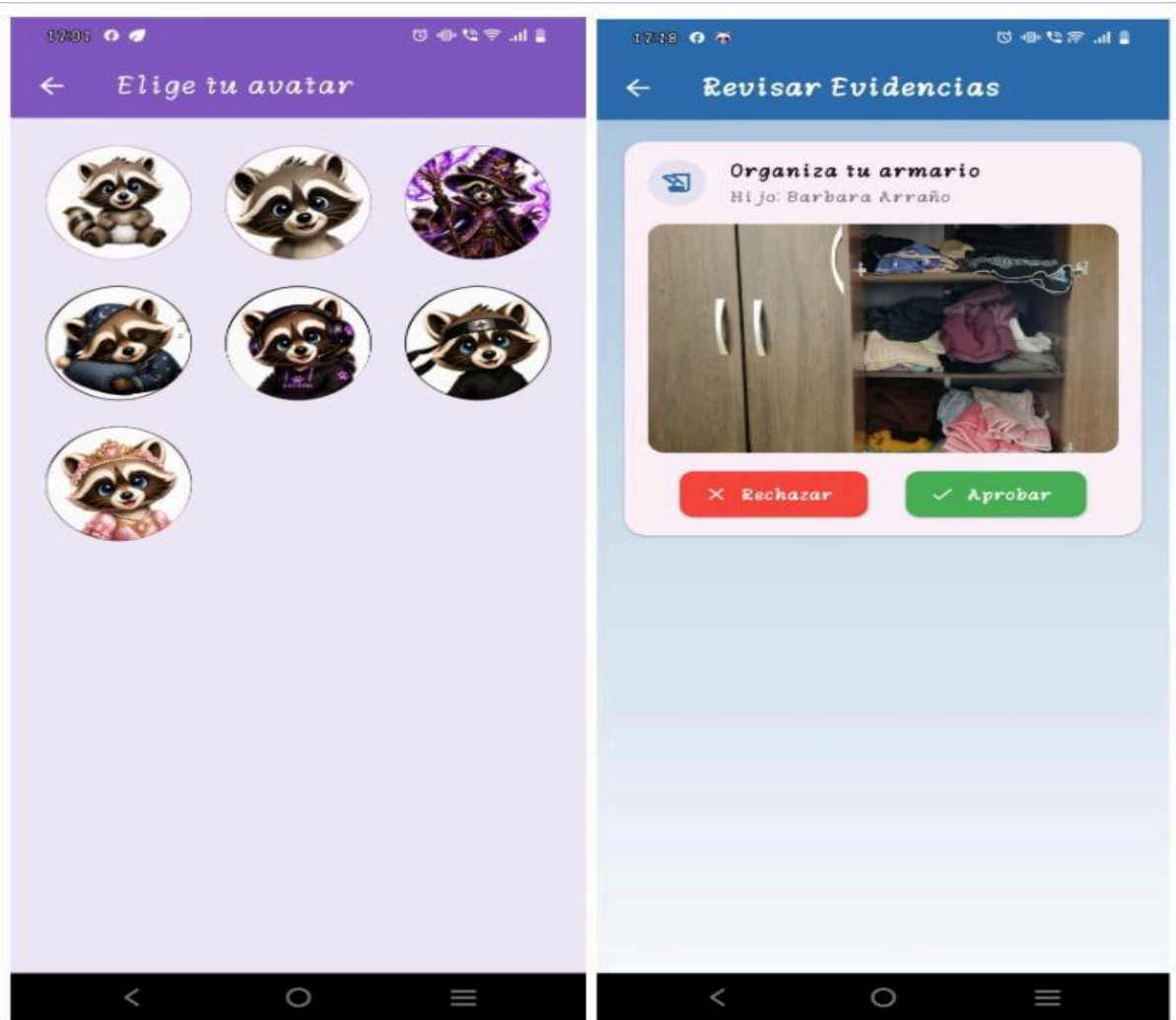
HU-09: Como padre, quiero crear recompensas con un nombre y un costo en puntos, para motivar a mi hijo a completar desafíos con metas concretas.



Mockups de la app: Inicio de sesión, Crear cuenta, Pantalla bloqueada, Panel de control del padre



Mockups de la app: configuración de bloqueo del hijo, desafíos, panel del hijo, tienda de premios



Mockups de la app: seleccione foto de perfil, revisión de evidencias enviadas

Propósito:

Mostrar la estructura visual de las pantallas del sistema antes de su implementación.

Qué representa:

- Pantalla de Inicio de sesión
- Pantalla de registro de usuario
- Pantalla de bloqueo de aplicaciones
- Pantalla de desafíos
- Subida de evidencia (foto)
- Panel parental
- Gestión de recompensas y canjes
- Selección de avatar
- Evolución del avatar
- Foto de perfil del hijo

Relación con el problema:

Permite entender cómo interactúan el padre y el hijo dentro del sistema de control parental y el flujo de uso de la aplicación.

Relación con los objetivos:

Define la estructura de la interfaz necesaria para implementar el control parental, la gamificación y la comunicación entre usuarios.

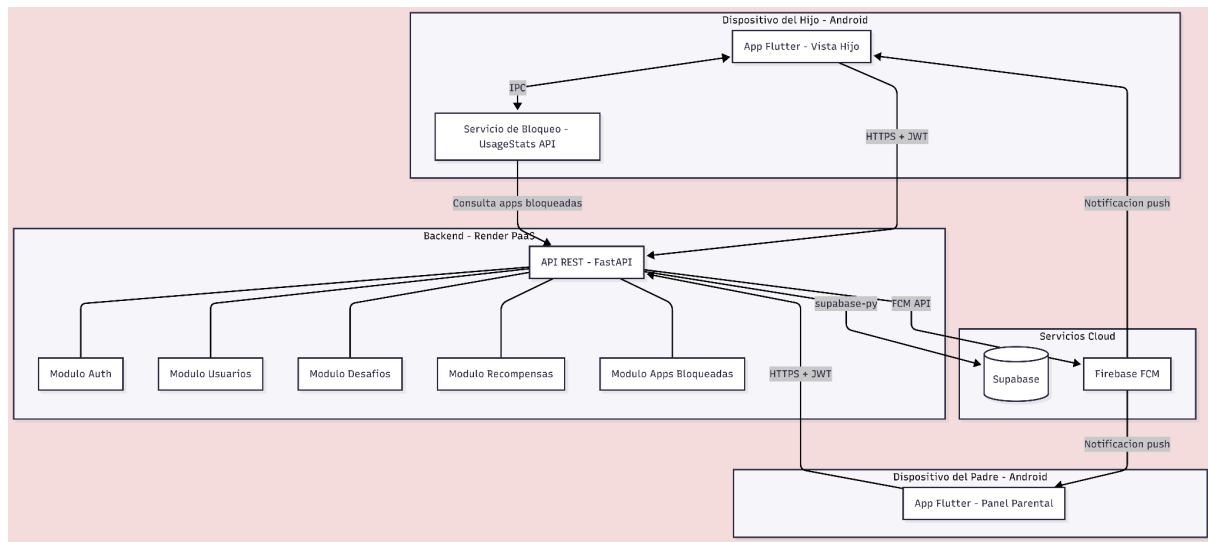


Diagrama de arquitectura (alto nivel).

Propósito:

Mostrar cómo se estructuran los componentes principales del sistema y cómo interactúan entre sí.

Qué representa:

- Aplicaciones móviles (padre e hijo)
- Backend/API
- Base de datos en Supabase
- Firebase FCM para notificaciones push y UsageStatsManager nativo de Android para monitoreo de uso de aplicaciones en el dispositivo del hijo

Relación con el problema:

Permite entender cómo el sistema controla el uso de aplicaciones, gestiona desafíos y comunica eventos entre padre e hijo.

Relación con los objetivos:

Define la base tecnológica necesaria para implementar el control parental, la gamificación y la comunicación en tiempo real.

13.Arquitectura de software y patrones de diseño

La arquitectura de MapacheSecure se ha diseñado bajo un modelo Cliente-Servidor con un enfoque de Arquitectura Limpia (Clean Architecture). Esta estructura garantiza que la lógica de negocio (desafíos, puntos y reglas de bloqueo) esté desacoplada de los servicios externos, permitiendo que el sistema sea testeable y escalable.

13.1. Arquitectura de N-Capas (Monolito Modular)

El sistema se organiza en capas con responsabilidades estrictamente separadas, tanto en el cliente como en el servidor:

- **Capa de Presentación (Flutter Client):** Gestiona la interfaz de usuario reactiva. Utiliza widgets que responden a cambios de estado y se comunica con las APIs nativas de Android (vía *Platform Channels*) para ejecutar el servicio de bloqueo de aplicaciones.
- **Capa de Dominio y Aplicación (FastAPI Backend):** Es el núcleo de la solución. Organizada en Routers (entrada de datos), Services (lógica de negocio y validación de retos) y Repositories (gestión de persistencia).
- **Capa de Infraestructura (Servicios Cloud):** Delegada a Supabase para la persistencia de datos y Firebase para las notificaciones push

13.2. Patrones de Diseño Implementados

Se han seleccionado patrones que optimizan el flujo de datos y la mantenibilidad del código:

Patrón Provider: Utilizado en Flutter para gestionar el estado de la aplicación. Permite separar la UI de la lógica de temas y sesiones, actualizando las vistas reactivamente ante cambios de estado.

Patrón Repository: Implementado en ambos extremos.

- En **Flutter**, centraliza las llamadas a la API de FastAPI.
- En el **Backend**, encapsula las consultas a Supabase, permitiendo que la lógica de negocio no dependa de la base de datos específica.

Patrón Observer: Utilizado mediante Streams en Flutter y Firebase Cloud Messaging en el sistema general. El dispositivo del padre "observa" los cambios en el estado del hijo, recibiendo notificaciones push inmediatas cuando se completa un desafío.

Patrón DTO (Data Transfer Object): Implementado mediante Pydantic en FastAPI. Define contratos estrictos para el intercambio de información (JSON), asegurando que los datos enviados desde Flutter sean validados antes de procesar cualquier lógica de puntos.

13.3. Justificación de la Arquitectura

Esta arquitectura es la más adecuada para el equipo por tres factores: Modularidad, Reactividad y Seguridad

- **Modularidad:** La división en routers/services/repositories nos permite poder trabajar en módulos independientes sin generar conflictos de código.
- **Reactividad:** Flutter y FastAPI (asíncrono) permiten que la experiencia de bloqueo y desbloqueo ocurra en menos de 500 ms, cumpliendo con los requisitos no funcionales de rendimiento.
- **Seguridad:** La separación entre capas garantiza que los secretos (SUPABASE_KEY, FCM_SERVER_KEY) nunca estén expuestos en el código. La autenticación JWT valida cada petición antes de acceder a los datos, protegiendo la información de los menores.

13.4. Representación Visual de la Arquitectura

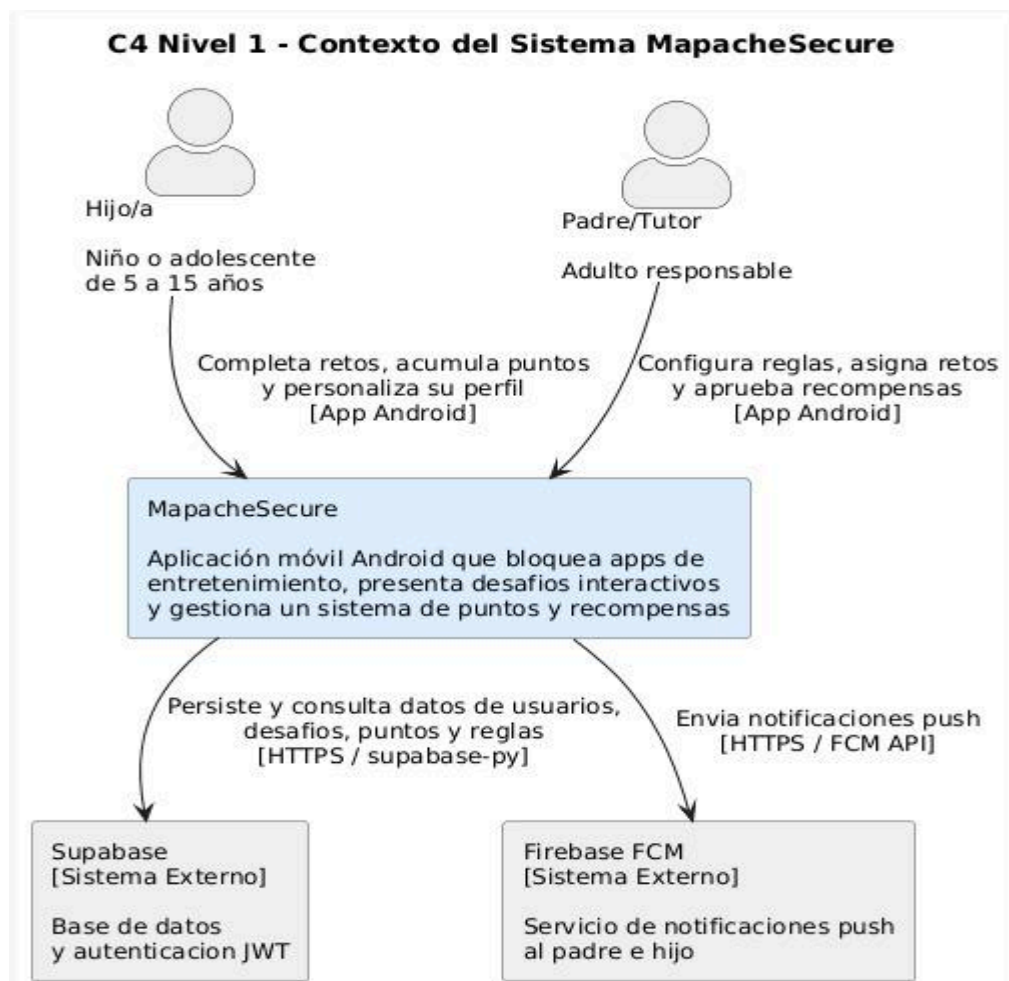


Diagrama C4 nivel 1: Contexto del sistema Mapacheseecure

C4 Nivel 1 - Contexto del Sistema

Este diagrama representa el ecosistema de MapacheSecure a alto nivel. Muestra la interacción de los actores principales (Hijo/a y Padre/Tutor) con el sistema central. Se destaca la dependencia de servicios externos críticos: Supabase para la persistencia de datos y gestión de identidad, y Firebase FCM para el despacho de notificaciones push, estableciendo un flujo de comunicación bidireccional donde el sistema central actúa como

mediador entre las reglas de control parental, la gamificación mediante desafíos y recompensas, y el comportamiento del usuario menor.

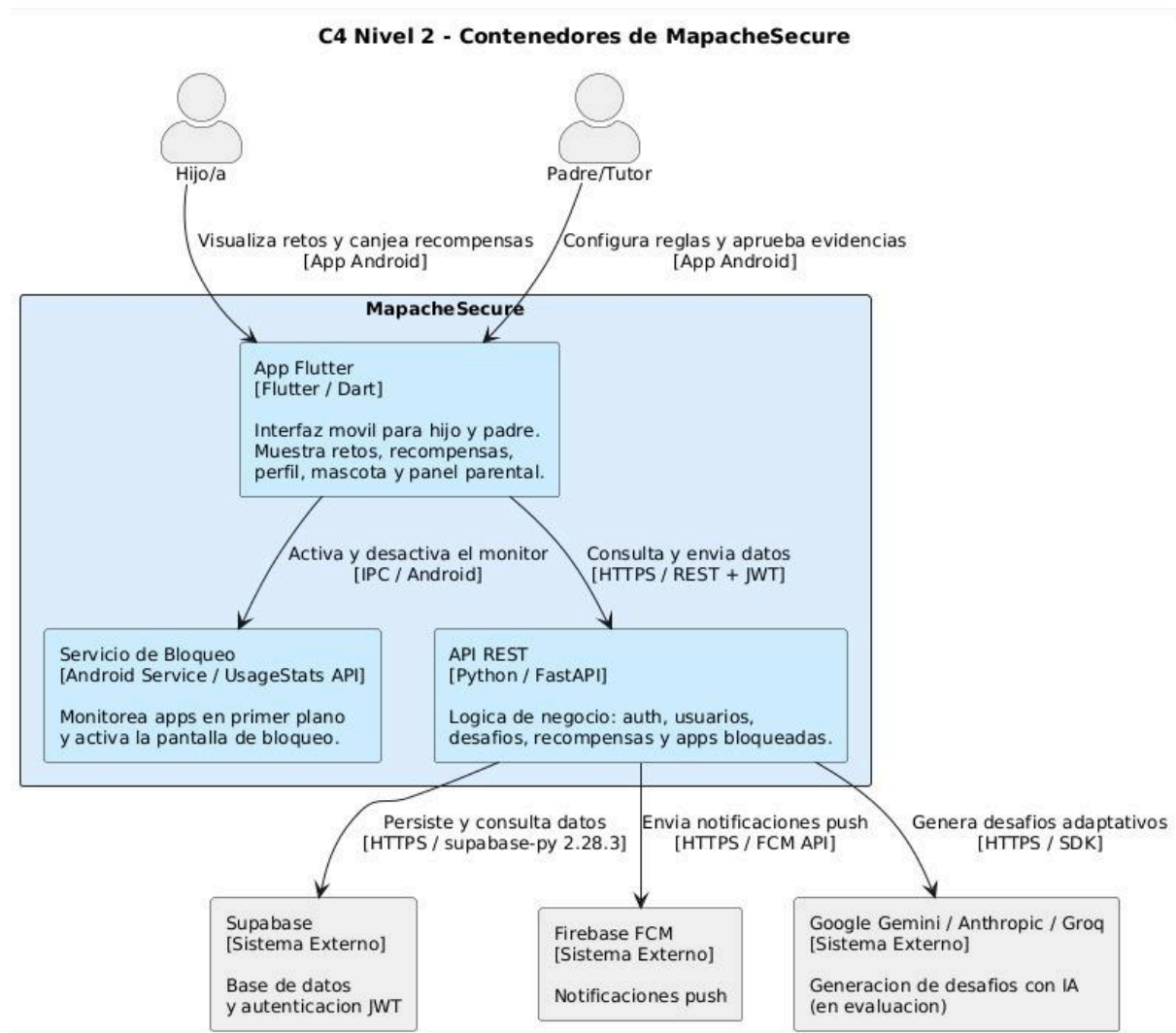


Diagrama C4 nivel 2: Contenedores de Mapacheseccure

C4 Nivel 2 - Contenedores

Este diagrama detalla la arquitectura interna del sistema, desglosada en componentes modulares. Se observa la App Flutter como interfaz principal, la cual se comunica mediante IPC con el BloqueoService (módulo nativo Android) para la intercepción de aplicaciones. El núcleo del negocio reside en la API REST (FastAPI), que coordina la lógica a través de diversos módulos especializados (Auth, Usuarios, Desafíos, Recompensas, Apps e IA) y gestiona la integración segura con las APIs externas de Supabase y Firebase mediante protocolos HTTPS y autenticación JWT.

14.Tecnologías, lenguaje y dependencias (stack definitivo)

Capa	Tecnología	Versión
App móvil	Flutter	3.41.7(Dart 3.11.5)
Backend / API	Python + FastAPI	Python 3.11
Base de datos	Supabase	—
Autenticación	Supabase Auth (JWT)	—
Notificaciones push	Firebase Cloud Messaging (FCM)	—
Hosting backend	Render (PaaS)	—
Cliente Supabase	supabase-py	2.28.3
Validación de datos	Pydantic	2.12.5
Servidor ASGI	Uvicorn	0.44.0
Gestión de proyecto	Trello (SCRUM)	—

Stack Tecnológico Definitivo

Librería	Versión
fastapi	0.135.3
uvicorn	0.44.0
supabase	2.28.3
python-dotenv	1.2.2
pydantic	2.12.5
google-genai	1.72.0
anthropic	0.94.0
firebase-admin	6.5.0
groq	0.28.0
resend	2.10.0
python-multiparte	0.0.20

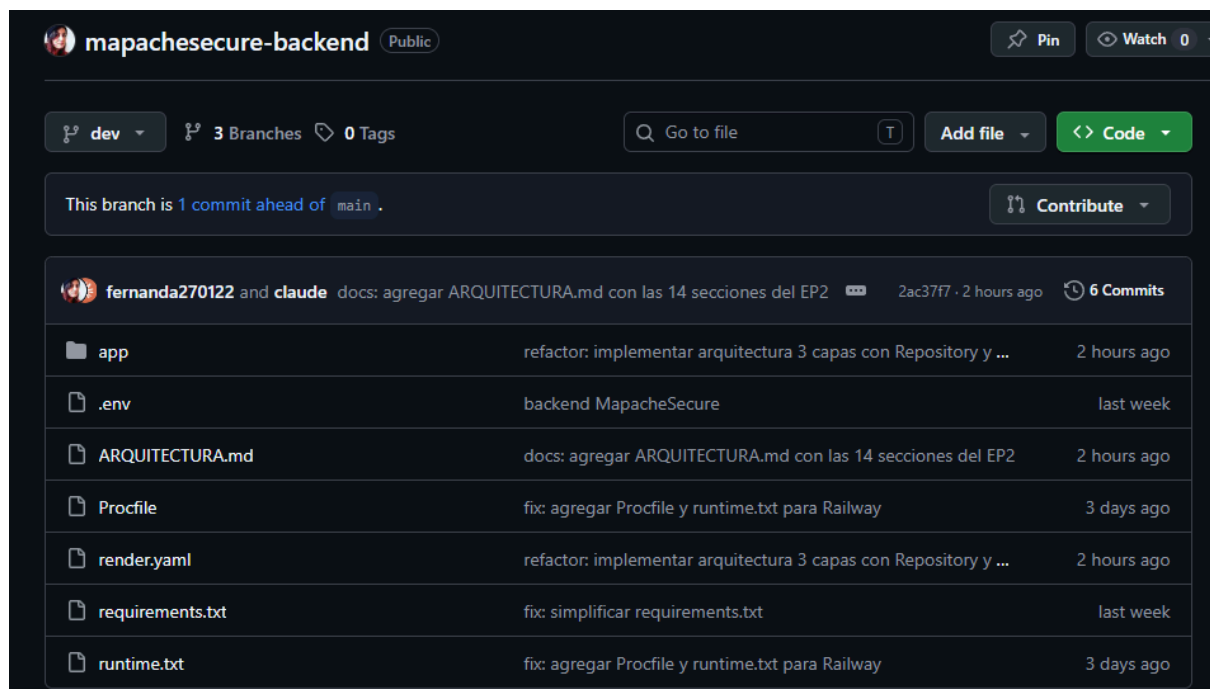
Dependencias del Backend (requirements.txt)

Estándares de Código

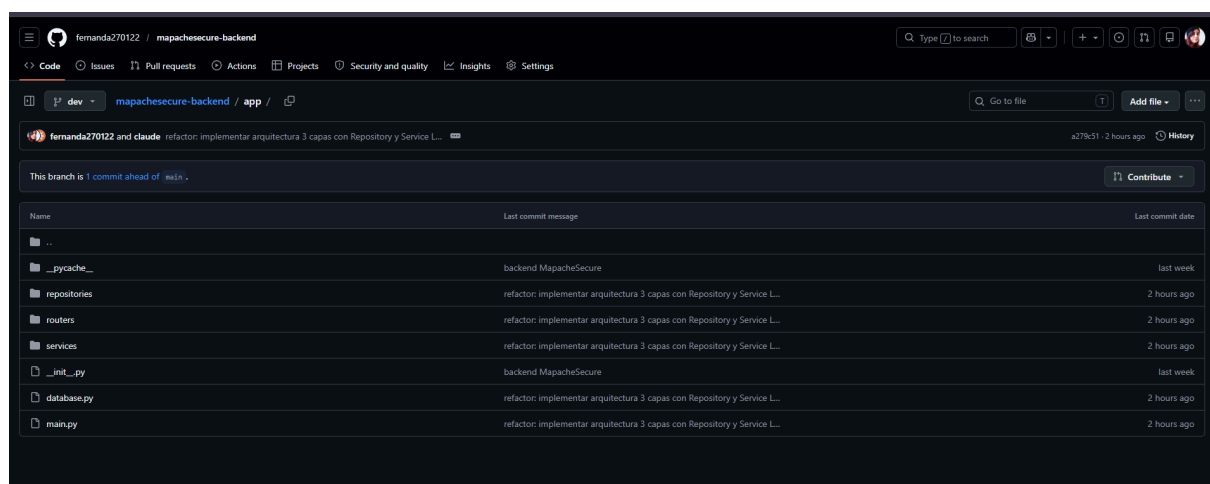
- Nomenclatura: snake_case para variables y funciones en Python; camelCase en Flutter.
- Estructura backend: routers / services / repositories (arquitectura en 3 capas).

- Commits: mensajes descriptivos con prefijos feat:, fix:, docs:, refactor:.
- Secretos: nunca en el código. Siempre en .env (local) o variables de entorno en Render (producción).
- Repositorio: github.com/fernanda270122/mapachese-secure-backend
- Ramas: main (producción) / dev (integración) / feature/* / fix/*
 - Link al repositorio.

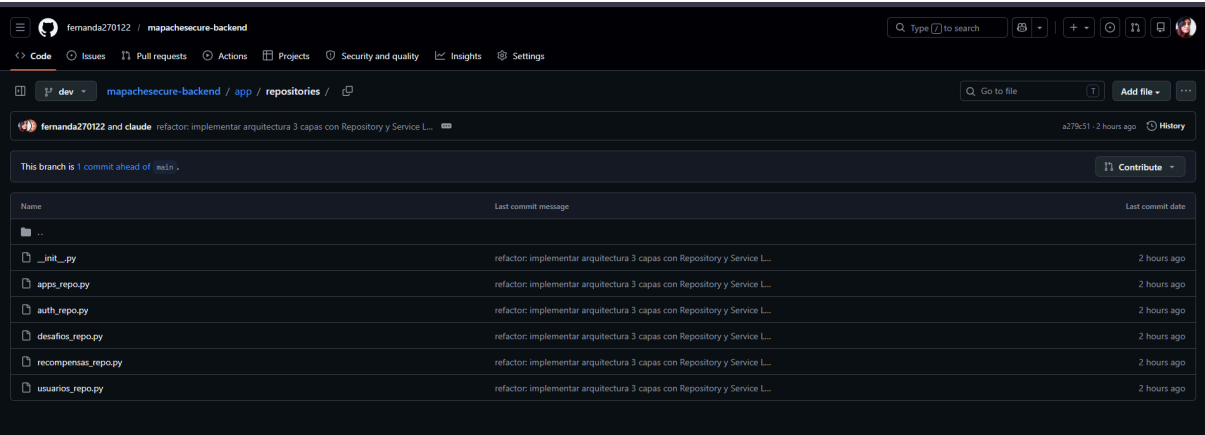
<https://github.com/fernanda270122/mapachese-secure-backend.git>



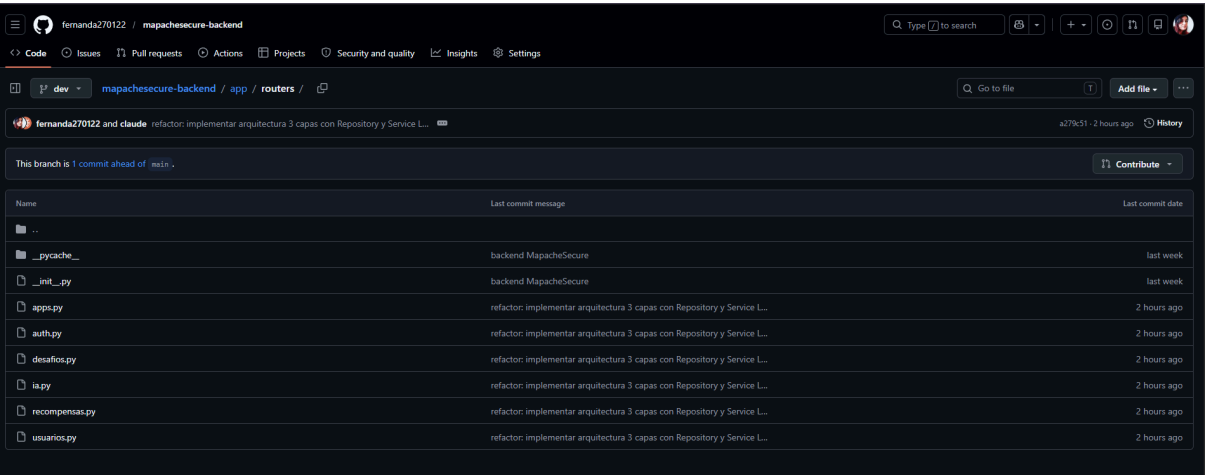
Github Mapachese-secure-backend



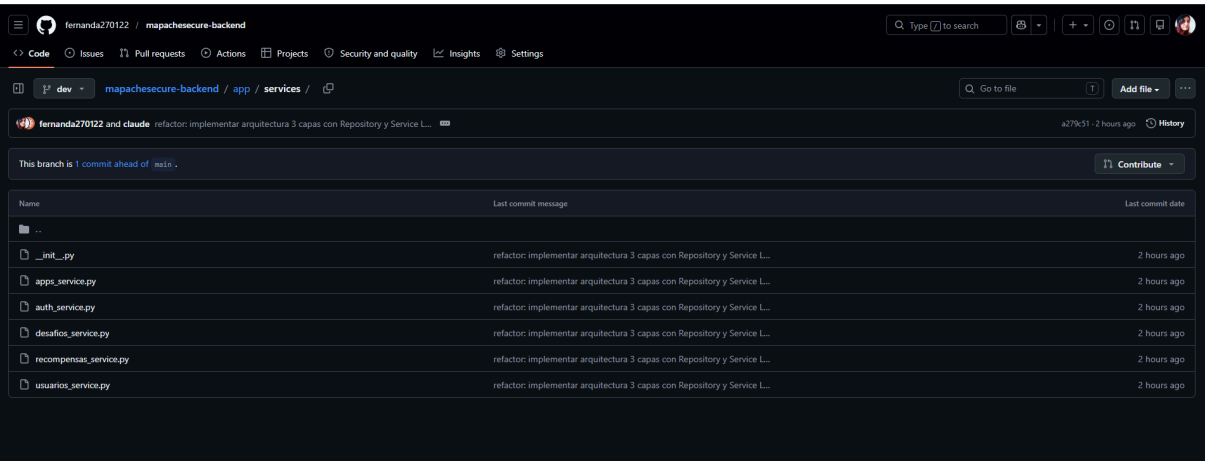
Github app



Github carpeta repositorio



Github carpeta Routes



Github carpeta Services

15. Configuración de servidor de producción (paso a paso)

Plataforma Cloud

El backend se despliega en Render (PaaS) con CI/CD automático desde GitHub. La base de datos y la autenticación se gestionan con Supabase (DBaaS). Ambas plataformas ofrecen plan gratuito suficiente para el desarrollo del semestre.

Requisitos del Servidor

- Runtime: Python 3.11.0
- Puerto: variable \$PORT asignada dinámicamente por Render
- Variables de entorno:
 - SUPABASE_URL,
 - SUPABASE_KEY
 - ANTHROPIC_API_KEY
 - GOOGLE_API_KEY
 - GROQ_API_KEY
 - RESEND_API_KEY
 - FCM_SERVER_KEY
- Comando de inicio: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- Archivo de configuración: `render.yaml` en la raíz del repositorio

Paso a Paso — Backend en Render

1. Crear cuenta en render.com y conectar la cuenta de GitHub.
2. New Web Service → conectar repositorio `mapachese-secure-backend` → rama `main`.
3. Configurar: Runtime = Python, Build Command = `pip install -r requirements.txt`, Start Command = `uvicorn app.main:app --host 0.0.0.0 --port $PORT`.
4. En Environment Variables, agregar todas las variables de entorno necesarias con los valores del proyecto Supabase.
5. Clic en Deploy. Render instala dependencias y levanta el servidor automáticamente.
6. Verificar que la API responde accediendo a `/docs` (Swagger UI) en la URL pública de Render.
7. Cada push a `main` dispara un deploy automático.

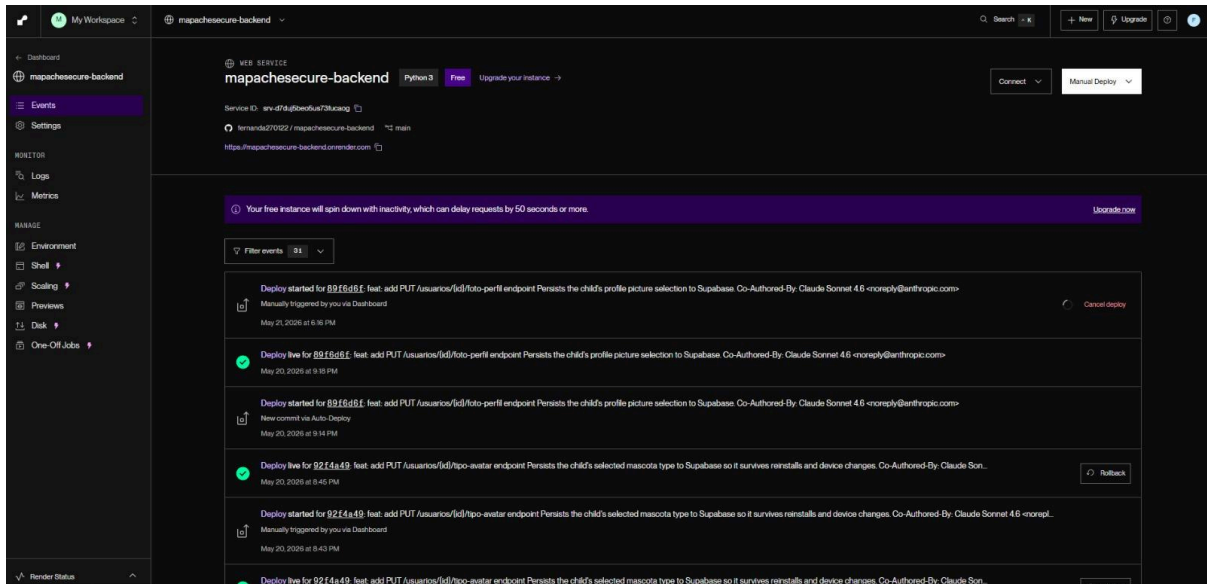
<https://dashboard.render.com/web/srv-d7duj5beo5us73fucaog>

<https://mapachese-secure-backend.onrender.com>

Paso a Paso — Base de Datos en Supabase

1. Crear proyecto en supabase.com.
2. Crear tablas: `usuarios`, `desafios`, `desafios_completados`, `apps_bloqueadas`, `recompensas`, `canjes`.

3. Activar Supabase Auth para registro y login con email/password.
4. Ir a Settings → API y copiar Project URL (SUPABASE_URL) y anon key (SUPABASE_KEY).
5. Pegar esas claves como variables de entorno en Render.



Despliegue en Render

<https://supabase.com/dashboard/project/xmsiydjwqvuzzyykaidgj>

Nota sobre disponibilidad gratuita

El plan gratuito de Render duerme el servicio tras 15 minutos de inactividad. Se mitiga configurando un ping automático con cron-job.org cada 10 minutos para mantener el servidor activo durante la demo.

16.Estado de avance del desarrollo (evidencia de producto funcional)

Funcionalidades Implementadas

1. Autenticación

Registro y login con Supabase Auth, generación de JWT, vinculación padre-hijo, logout.

Evidencia: POST /auth/registro y POST /auth/login respondiendo con token y perfil. Probado en Swagger UI (/docs).

2. Gestión de Usuarios

Creación de perfiles, obtención de usuario por ID, listado de hijos por padre_id.

Evidencia: GET /usuarios/, POST /usuarios/, GET /usuarios/{padre_id}/hijos.

3. Módulo de Desafíos

Listado de desafíos por tipo, completar desafío con o sin foto, validación por el padre, consulta de puntos.

Evidencia: POST /desafíos/completar, PUT /desafíos/validar/{id}, GET /desafíos/puntos/{hijo_id}.

4. Sistema de Recompensas

Creación de recompensas por el padre, canje de puntos, historial de canjes.

Evidencia: POST /recompensas/, POST /recompensas/canjear, GET /recompensas/historial/{hijo_id}.

5. Apps Bloqueadas

Agregar y eliminar apps por hijo. Endpoint de verificación consultado por la app Android.

Evidencia: GET /apps/{hijo_id}, POST /apps/, GET /apps/verificar/{hijo_id}/{package_name}.

6. Avatar y evolución de mascota — Implementada

Sistema de gamificación donde el hijo selecciona un avatar entre 6 opciones (mago, dormilón, gamer, ninja, samurái, princesa) y lo hace subir de nivel acumulando puntos mediante desafíos completados. Niveles desbloqueables en 500, 1100, 1900, 2900, 4100 y 5500 puntos. Evidencia: PUT /usuarios/{id}/tipo-avatar, GET /desafíos/puntos/{hijo_id}.

7. Foto de perfil del hijo — Implementada

El hijo puede personalizar su foto de perfil eligiendo entre imágenes predeterminadas de Raccu. Evidencia: PUT /usuarios/{id}/foto-perfil.

Funcionalidades En Progreso

1. Notificaciones Push (FCM) — En progreso

Integración de Firebase Cloud Messaging para alertas al padre cuando el hijo completa un desafío.

2. App Android — En progreso

Pantalla de bloqueo mediante UsageStatsManager con polling cada 500 ms para detectar la app en primer plano. Integración con la API vía HTTP usando api service.dart.

Calidad y Seguridad

- Todos los endpoints retornan códigos HTTP correctos: 200, 404, 500.
- Las respuestas de error siempre incluyen el campo detail con descripción.
- Secretos nunca en el código fuente. Uso de .env local y variables de entorno en Render.
- Código refactorizado en 3 capas: routers / services / repositories.
- Control de versiones con ramas main, dev, feature/* y fix/*.

La pauta evalúa que el desarrollo avance con tecnologías adecuadas y buenas prácticas, cumpliendo funcionalidades aprobadas.

17.Integraciones necesarias

El ecosistema de **MapacheSecure** se sustenta en la integración de servicios de terceros que permiten delegar responsabilidades críticas como la seguridad, la persistencia y la comunicación en tiempo real. Esto permite que el equipo se enfoque en la lógica de negocio y la gamificación, asegurando un estándar profesional en la gestión de datos.

17.1. Detalle de Integraciones Existentes y Planificadas

1. Supabase — Base de Datos y Autenticación

Es el pilar de persistencia y seguridad del sistema. Supabase actúa como el backend-as-a-service (BaaS) que provee una base de datos y un sistema de gestión de identidades.

- **Qué hace:** Persiste en todas las entidades del sistema (usuarios, desafíos, puntos, configuración de apps bloqueadas) y gestiona el ciclo de vida de las sesiones mediante tokens JWT.
- **Datos de entrada:** Credenciales de usuario (Email/Password) para autenticación; estructuras JSON para operaciones CRUD (Create, Read, Update, Delete).
- **Datos de salida:** Tokens JWT de sesión, perfiles de usuario y registros de tablas solicitados por la API.
- **Librería técnica:** supabase-py versión 2.28.3.

2. Firebase Cloud Messaging (FCM) — Notificaciones Push

Este servicio es el canal de comunicación asíncrona entre el servidor y los dispositivos móviles, eliminando la necesidad de que la app realice consultas constantes (polling) al servidor.

- **Qué hace:** Envía notificaciones push al padre cuando el hijo completa un desafío y alertas de incentivo al dispositivo del hijo.
- **Datos de entrada:** Device Token (identificador único del celular), título, cuerpo del mensaje y metadatos de navegación.
- **Datos de salida:** ID de mensaje de Firebase y estado de confirmación de entrega.
- **Estado:** Pendiente de implementación técnica para el Sprint 3.

3. Google Gemini / Anthropic Claude — Módulo IA/ Groq — Módulo IA

Integración orientada a la generación de contenido dinámico para evitar la fatiga del usuario por retos repetitivos.

- **Qué hace:** Generación adaptativa de desafíos (trivias, lecturas, retos lógicos) basados en la edad del menor y su historial de cumplimiento.
- **Librerías preparadas:** google-genai (1.72.0) y anthropic (0.94.0), groq (0.28.0)
- **Estado:** En fase de evaluación y prototipado lógico.

4. Resend — Envío de Emails

Qué hace: Envía emails transaccionales durante el flujo de autenticación, incluyendo recuperación de contraseña y verificación de identidad.

Datos de entrada: Dirección de email del usuario, asunto y cuerpo del mensaje.

Datos de salida: Confirmación de entrega del email.

Librería técnica: resend 2.10.0

Estado: Integrado y funcionando en producción.

17.2. Consideraciones de Seguridad y Protección de Datos

Dada la naturaleza del proyecto y el manejo de datos de menores, la seguridad en las integraciones es la máxima prioridad:

- **Gestión de Secretos:** Todas las credenciales críticas (SUPABASE_KEY, FCM_SERVER_KEY, API Keys de IA) se gestionan estrictamente como Variables de Entorno en la plataforma de hosting (Render). Se prohíbe el almacenamiento de estas claves en archivos de texto plano o en el repositorio de GitHub.
- **Autenticación por Token:** Todas las llamadas a la API de FastAPI requieren un encabezado de autorización Bearer con un JWT válido emitido por Supabase. El backend valida la firma del token antes de permitir el acceso a los datos.
- **Configuración de Cliente:** La integración con la app móvil requiere el archivo google-services.json, el cual está configurado para permitir la comunicación segura solo desde el paquete de aplicación (com.mapacheseccure) firmado por el equipo.

17.3. Flujo de Datos y Comunicación entre Servicios

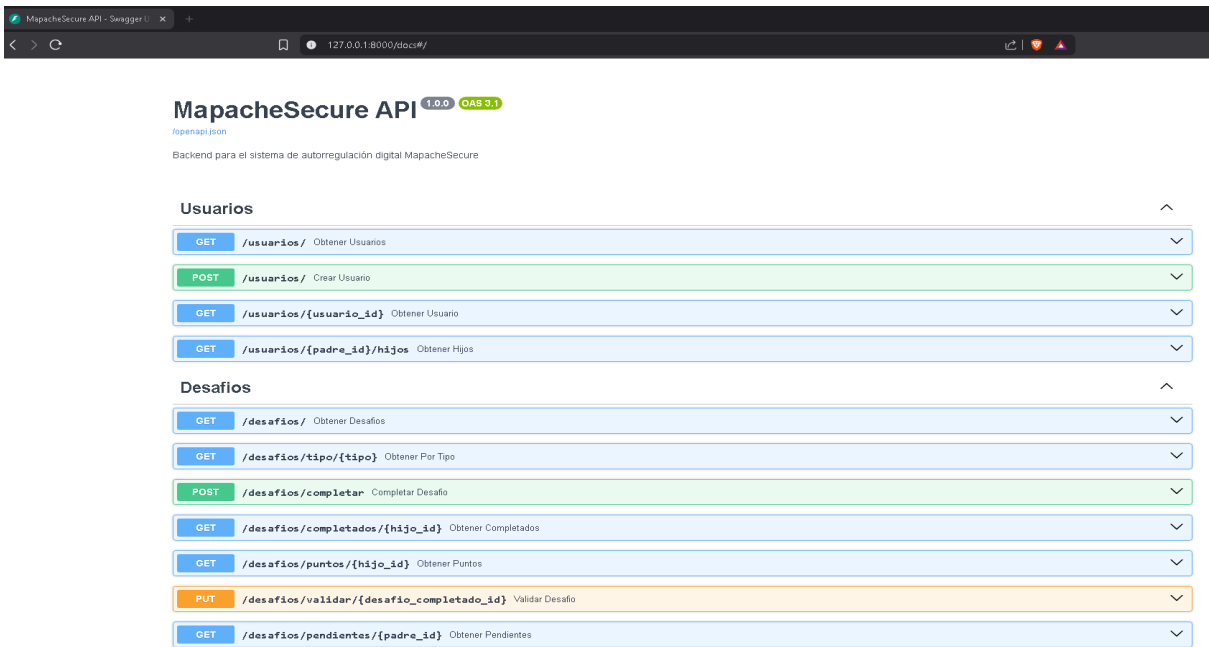
Para que la integración sea efectiva, el sistema sigue un flujo de información bidireccional donde el backend actúa como mediador central:

- **Flujo de Autenticación:** El cliente (Flutter) envía credenciales a la API; esta consulta a Supabase Auth, el cual retorna un JWT. Este token es almacenado localmente en el dispositivo para firmar cada petición posterior.
- **Flujo de Notificaciones:** Cuando el DesafiosService en FastAPI detecta que un desafío ha sido validado, gatilla una petición hacia los servidores de Firebase (FCM) enviando el *payload* del mensaje. Firebase se encarga de despertar el servicio en segundo plano del dispositivo del padre para mostrar la alerta.
- **Generación de Contenido (IA):** El backend envía un *prompt* estructurado con el nivel de dificultad y edad del menor. La IA responde con un objeto JSON que el backend procesa, guarda en la base de datos y finalmente entrega al móvil para ser renderizado como un desafío interactivo.

17.4. Evidencias de Pruebas de Integración

Como respaldo del avance técnico y la correcta comunicación entre los componentes, se presentan las siguientes evidencias:

1. Documentación y Contrato de API (Swagger UI)



2. Validación de Repuesta de Datos

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/usuarios/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/usuarios/
```

Server response

Code	Details
200	Response body <pre>[{ "id": "93fe88fa-8604-440f-8516-a0f4faef728f", "email": "elena@mapachecure.com", "nombre": "Elena", "rol": "padre", "padre_id": null, "edad": 36, "created_at": "2026-04-12T16:53:20.9964" }, { "id": "19562a00-5eaa-4407-badf-46f74532180c", "email": null, "nombre": "mateo", "rol": "hijo", "padre_id": null, "edad": 10, "created_at": "2026-04-12T16:55:36.205969" }]</pre> Response headers <pre>content-length: 335 content-type: application/json date: Tue, 21 Apr 2026 02:49:33 GMT server: uvicorn</pre>

1. Supabase — Base de Datos y Autenticación

Qué hace: persiste todos los datos del sistema (usuarios, desafíos, puntos, apps bloqueadas) y gestiona el registro y login con tokens JWT.

Librería: supabase-py 2.28.3

Datos de entrada: credenciales de usuario para auth; datos de entidad para CRUD.

Datos de salida: tokens JWT, datos de usuario y registros de tablas.

Seguridad: SUPABASE_URL y SUPABASE_KEY almacenadas como variables de entorno, nunca en el repositorio.

Estado: integrado y funcionando en producción (Render).

2. Firebase Cloud Messaging (FCM) — Notificaciones Push

Qué hace: envía notificaciones push al padre cuando el hijo completa un desafío, y alertas de logro al hijo.

Datos de entrada: token FCM del dispositivo destino, título y cuerpo del mensaje.

Datos de salida: confirmación de entrega de la notificación.

Seguridad: FCM Server Key almacenada como variable de entorno.

Consideraciones: requiere configurar google-services.json en la app Android y habilitar FCM en la consola de Firebase.

Estado: pendiente de implementación (sprint 3).

3. Google Gemini / Anthropic Claude — Módulo IA (planificado)

Qué hace: generación dinámica de desafíos adaptativos según el perfil del hijo (edad, historial de desafíos completados).

Las dependencias google-genai y anthropic están incluidas en requirements.txt como preparación para su implementación.

Estado: en evaluación. No está implementado en el sprint actual.

18. Estrategia y enfoque de calidad

Atributos de calidad evaluados

En Raccu, la calidad se define como la capacidad de la aplicación de proteger efectivamente a los niños dentro de los horarios configurados por sus padres, sin interrupciones ni vulnerabilidades explotables. Los atributos priorizados son:

1. Funcionalidad: Que cada módulo cumpla su propósito: bloqueo de apps, desafíos, mascota, perfil, notificaciones e IA.
2. Fiabilidad: El guardián de bloqueo debe operar en background de forma continua y sin caídas.
3. Seguridad: Las sesiones deben estar correctamente protegidas; la pantalla de bloqueo no debe ser eludible.
4. Rendimiento: El parsing de datos y las búsquedas en memoria deben responder dentro de umbrales aceptables incluso con volúmenes altos.
5. Usabilidad: Los flujos críticos (login, agregar hijo, seleccionar avatar) deben completarse sin ambigüedades visuales.

Niveles y tipos de prueba

1. Se aplica una estrategia en capas, de lo más granular a lo más completo:
2. Pruebas unitarias: verifican modelos, providers y servicios de forma aislada.
3. Pruebas de integración: validan que los módulos colaboren correctamente (AuthService + SharedPreferences, providers entre si).
4. Pruebas funcionales: recorren flujos completos desde la perspectiva del usuario (login, bloqueo, avatar, recuperación de contraseña).
5. Pruebas de humo (smoke): comprueban en dispositivo real que las pantallas críticas arrancan tras cada compilación.
6. Pruebas de regresión: aseguran que cambios recientes no rompan comportamientos previos.
7. Pruebas de rendimiento/carga: miden tiempo de procesamiento con volúmenes altos de datos.
8. Pruebas de seguridad: verifican autenticación, control de sesión e impermeabilidad de la pantalla de bloqueo.

Enfoque de prueba

La totalidad de las pruebas está automatizada con el framework `flutter_test` (pruebas unitarias, integración, funcionales, regresión, carga y seguridad) y con `integration_test` para las pruebas de humo en dispositivo. Los datos de prueba son sintéticos (objetos mock y SharedPreferences simulados), no se utilizan credenciales ni endpoints reales durante la ejecución. El ambiente de ejecución es el entorno local de desarrollo con Flutter SDK 3.x (Dart ^3.11.5).

Definition of Done

- El 100% de las pruebas unitarias, de integración, funcionales, de regresión, de carga y de seguridad pasan en verde.
- Las 9 pruebas de humo en un dispositivo físico pasan sin errores visuales.
- No existen defectos abiertos de severidad alta o bloqueante.
- El Código está versionado en la rama correspondiente y el pipeline de pruebas se ejecuta sin fallas.

19. Plan de pruebas

Alcance

El plan cubre el frontend Android de Raccu (proyecto mapacheseccure_app, Flutter/Dart) y el backend (proyecto mapacheseccure-backend, FastAPI/Python). El frontend se prueba con flutter_test e integration_test. El backend se prueba con pytest, usando mocks de Supabase para no depender de credenciales reales ni de red durante la ejecución.

Funcionalidades a probar

- Autenticación de usuarios (login, logout, recuperación de contraseña)
- Registro y gestión de hijos
- Bloqueo de apps por horario (guardian en background)
- Gestión de reglas (panel del padre)
- Sistema de desafíos
- Sistema de mascota y avatares
- Foto de perfil del hijo
- Asistente IA para padres
- Notificaciones push (FCM)
- Dashboard del hijo (actividad y tiempo de pantalla)

Criterios de aceptación

- 100% de las pruebas unitarias, funcionales, de integración, regresión, carga y seguridad aprobadas.
- 4/4 pruebas de humo en dispositivo físico aprobadas.
- 0 defectos críticos (bloqueo de funcionalidad principal) abiertos al cierre.

Criterios de suspensión y reanudación

Las pruebas se suspenden si un defecto impide la compilación o el arranque de la aplicación. Se reanuda una vez corregido el defecto y validado por al menos un integrante del equipo.

Integrante	Rol en pruebas	Responsabilidad principal
Fernanda Arrano	QA Lead / Desarrolladora	Pruebas unitarias, funcionales y humo. Redacción del informe.
Javier Vieytes	Desarrollador / Tester	Pruebas de integración y regresión. Revisión de resultados.
Benjamin Castillo	Desarrollador / Tester	Pruebas de seguridad y carga. Configuración de ambiente.

Recursos y roles

Riesgo	Mitigación
Dispositivo no disponible para pruebas de humo	Usar emulador como respaldo; documentar diferencias.
Cambios de última hora en pantallas	Mantener smoke tests diagnósticos al contenido visual específico.
Dependencias de plugins Android (UsageStats, BackgroundService)	Mockear en pruebas unitarias; validar solo en dispositivo físico.

Riesgos y mitigación

20. Tipos de pruebas aplicadas

20.1 pruebas unitarias

Herramienta: flutter_test (SDK de Flutter)

Casos: 75 casos: 36 modelos + 10 providers + 21 servicios + 8 pantallas

Verifican que cada modelo (AppBloqueada, Desafio, PetModel, ReglaBloqueo, Recompensa, Usuario, AvatarType), provider (ActividadProvider, TemaProvider, TemaPadreProvider) y servicio (ApiService, AuthService, GuardianService, laService, NotificationService) se comporten correctamente de forma aislada. Usan SharedPreferences simulados y MockClient para no depender de red ni almacenamiento real. Esta es la capa con mayor cobertura del proyecto.

20.2 Pruebas de humo (smoke)

Herramienta: integration_test (SDK de Flutter) + dispositivo Android físico

Casos: 9 casos ejecutados en Vivo V2127 (Android 14)

Comprueban que cada una de las 9 historias de usuario críticas arranca y responde en dispositivo real: registro, login, agregar hijo, configurar hijo, desafíos del hijo, recompensas, mis desafíos, selección de avatar y pantalla de bloqueo. Se realizan 2 logins reales al inicio para obtener tokens; el resto usa datos inyectados o mocks HTTP.

20.3 Pruebas de integración

Herramienta: flutter_test (SDK de Flutter)

Casos: 4 casos

Validan la colaboración entre módulos internos: AuthService leyendo estado de sesión desde SharedPreferences, persistencia y recuperación de temas (TemaProvider y TemaPadreProvider) sin interferencia entre claves, y que la selección de avatar actualiza la clave correcta en SharedPreferences.

20.4 Pruebas funcionales

Herramienta: flutter_test + testWidgets

Casos: 4 casos

Recorren flujos completos como lo haría un usuario: login fallido con credenciales vacías, selección visual de avatar con confirmación de borde deepPurple, visualización de pantalla de bloqueo con horario real y navegación al flujo de recuperación de contraseña.

20.5 Pruebas de regresión

Herramienta: flutter_test

Casos: 4 casos

Aseguran que cambios recientes no rompan comportamientos consolidados: que el logout preserve preferencias de onboarding y paleta de color, que un ID de avatar invalido retorne el mago como fallback, que PetModel no falle con puntos extremos (0 o 99999) y que el estado de desafio en mayusculas sea interpretado correctamente.

20.6 Pruebas de rendimiento / carga

Herramienta: flutter_test + Stopwatch de Dart

Casos: 4 casos con umbrales de tiempo explícitos

Miden el tiempo de procesamiento bajo volúmenes altos: parsear 1.000 desafíos en menos de 500 ms, parsear 500 apps bloqueadas en menos de 300 ms, calcular tiempo total de actividad con 100 registros en menos de 200 ms, y 10.000 búsquedas de avatar por ID en menos de 1.000 ms. Todos los umbrales fueron superados con un margen amplio.

20.7 Pruebas de seguridad

Herramienta: flutter_test

Casos: 4 casos

Verifican autenticación y control de acceso: sin token, isLoggedIn retorna false; al eliminar el token la sesión se invalida; JSON con campos null no lanza excepción en Desafio.fromJson(); y la PantallaBloqueoScreen tiene canPop = false, impidiendo que el niño salga mediante el botón de retroceso.

20.8 Pruebas unitarias de backend

Herramienta: pytest + pytest-mock (Python)

Casos: 106 casos: autenticación, apps, canjes, desafíos, IA, recompensas, usuarios

Cubren la capa de servicios del backend FastAPI. Cada servicio se prueba de forma aislada mockeando el repositorio correspondiente, por lo que ningún test toca la base de datos Supabase ni la red. Se validan flujos exitosos, errores esperados (404, 400, 403, 500) y casos bordes como correos duplicados, canjes sin puntos suficientes y avatares inválidos. Resultado: 106/106 en verde.

21. Herramientas y software utilizados

Herramienta	Tipo de prueba / uso	Versión	Justificación
flutter_test	Unitarias, integración, funcionales, regresión, carga, seguridad	SDK Flutter 3.x	Framework oficial de Flutter. Incluye testWidgets, pump, pumpAndSettle y matchers. No requiere dependencia externa adicional.
integration_test	Pruebas de humo en dispositivo	SDK Flutter 3.x	Paquete oficial para pruebas en dispositivo físico o emulador. Permite simular interacciones reales sobre la UI compilada.
MockClient (http/testing)	Mock de API REST	http ^1.2.1	Intercepta llamadas HTTP y retorna respuestas simuladas sin contactar el backend real. Usado en servicios y smoke tests.
SharedPreferences fake	Persistencia simulada	shared_preferences ^2.3.2	setMockInitialValues() precarga valores en memoria para que los tests sean deterministas sin tocar disco.

flutter_screenutil	Soporte responsivo en smoke tests	^5.9.3	Necesario en integration_test para inicializar ScreenUtilnit antes de renderizar pantallas con tamaños responsivos.
Stopwatch (Dart core)	Medición de tiempo en pruebas de carga	Dart ^3.11.5	Clase nativa de Dart usada para medir tiempos de ejecución y comparar contra umbrales definidos en cada test.
pytest + pytest-mock	Pruebas unitarias de backend	pytest 9.0.3	Framework de pruebas de Python. pytest-mock proporciona la fixture mocker para reemplazar repositorios con objetos simulados sin tocar Supabase.
GitHub	Control de versiones	N/A	Repositorio centralizado. Las ramas main (producción) y dev (desarrollo) permiten integrar cambios de forma controlada.
VS Code + Dart extension	Entorno de desarrollo y ejecución	N/A	Permite correr flutter test directamente desde el editor con visualización de resultados en tiempo real.

22. Ejecución de pruebas y resultados

Resumen de ejecución

Las pruebas del frontend se ejecutaron con el comando flutter test --reporter expanded en el directorio raíz del proyecto mapachese_secure_app. Las pruebas de humo se ejecutaron sobre un dispositivo físico Vivo V2127 (Android 14, FuntouchOS) vía ADB. Las pruebas del backend se ejecutaron el 14 de junio de 2026 con el comando python -m pytest tests/ -v en el directorio mapachese_secure-backend, obteniendo 106/106 pruebas en verde en 0.82 segundos.

Tipo de prueba	Total casos	Exitosos	Fallidos	% Éxito
Unitarias Flutter (modelos)	36	36	0	100%
Unitarias Flutter (providers)	10	10	0	100%
Unitarias Flutter (servicios)	21	21	0	100%
Unitarias Flutter (pantallas)	8	8	0	100%
Unitarias Backend (pytest)	106	106	0	100%
Integración	4	4	0	100%

Funcionales	4	4	0	100%
Regresión	4	4	0	100%
Seguridad	4	4	0	100%
Carga / Rendimiento	4	4	0	100%
Humo (dispositivo físico)	9	9	0	100%
TOTAL	210	210	0	100%

Resultados por tipo de prueba

Pruebas unitarias (75 casos)

Se cubrieron 7 modelos (AppBloqueada, AvatarType, Desafio, PetModel, ReglaBloqueo, Recompensa, Usuario), 3 providers (ActividadProvider, TemaProvider, TemaPadreProvider) y 5 servicios (ApiService, AuthService, GuardianService, IaService, NotificationService), mas 2 pantallas (LoginScreen, AvatarScreen). Resultado: 75/75 exitosos. Tiempo aproximado de ejecución: 3 segundos.

Casos representativos:

- Recompensa.fromJson() maneja casteo seguro de float a int y valores nulos.
- ReglaBloqueo con horario válido se construye correctamente desde JSON.
- PetModel asigna nivel e imagePath correctos según umbrales de puntos.
- ActividadProvider.tiempoTotalPantalla() suma correctamente la lista en memoria.
- AuthService.getRol() lee la clave correcta desde SharedPreferences.

- NotificationService.registrarToken() controla token nulo sin lanzar excepcion.
- laService: POST /ia/generar retorna lista de desafíos correctamente estructurados.

Models:

```

import 'package:flutter_test/flutter_test.dart';
import 'package:sapachecore_app/models/app_bloqueada.dart';

void main() {
  // Grupo de pruebas unitarias para el modelo app_bloqueada.dart
  group('Pruebas unitarias para el modelo app_bloqueada', () {
    // Datos fijos para simular la respuesta exacta del backend (en estado caso)
    final Response jsonResponse = {
      'id': 'app_123',
      'tipo_id': '101',
      'nombre_app': 'YouTube',
      'package_name': 'com.google.android.youtube',
      'requiere_ubicacion': true,
      'fecha_creacion': '2023-05-04T12:00:00.000Z',
    };

    // Test
    test('1. Debería construir una instancia correcta desde un JSON (fromJson)', () {
      // Act: Convertimos el JSON al objeto de Dart
      final app = AppBloqueada.fromJson(jsonDecode(jsonResponse));

      // Assert: Verificamos que las propiedades se hayan asignado y transformado bien
      expect(app.id, 'app_123');
      expect(
        app.tipoId,
        '101',
      );
      expect(
        app.nombreApp,
        'YouTube',
      );
      expect(
        app.packageName,
        'com.google.android.youtube',
      );
      expect(
        app.requiereUbicacion,
        true,
      );
      expect(
        app.fechaCreacion,
        DateTime.parse('2023-05-04T12:00:00.000Z'),
      );
    });
  });
}

```

App_bloqueada_test.dart

```

import 'package:flutter_test/flutter_test.dart';
import 'package:sapachecore_app/models/avatar_type.dart';

void main() {
  // Grupo de pruebas unitarias para AvatarType y AvatarTypes
  group('Pruebas unitarias para AvatarType y AvatarTypes', () {
    // Test
    test('1. El getter previos debe devolver la imagen del primer nivel (índice 0)', () {
      // Act: Creamos una instancia de AvatarTypes
      final avatar = AvatarTypes.mags;

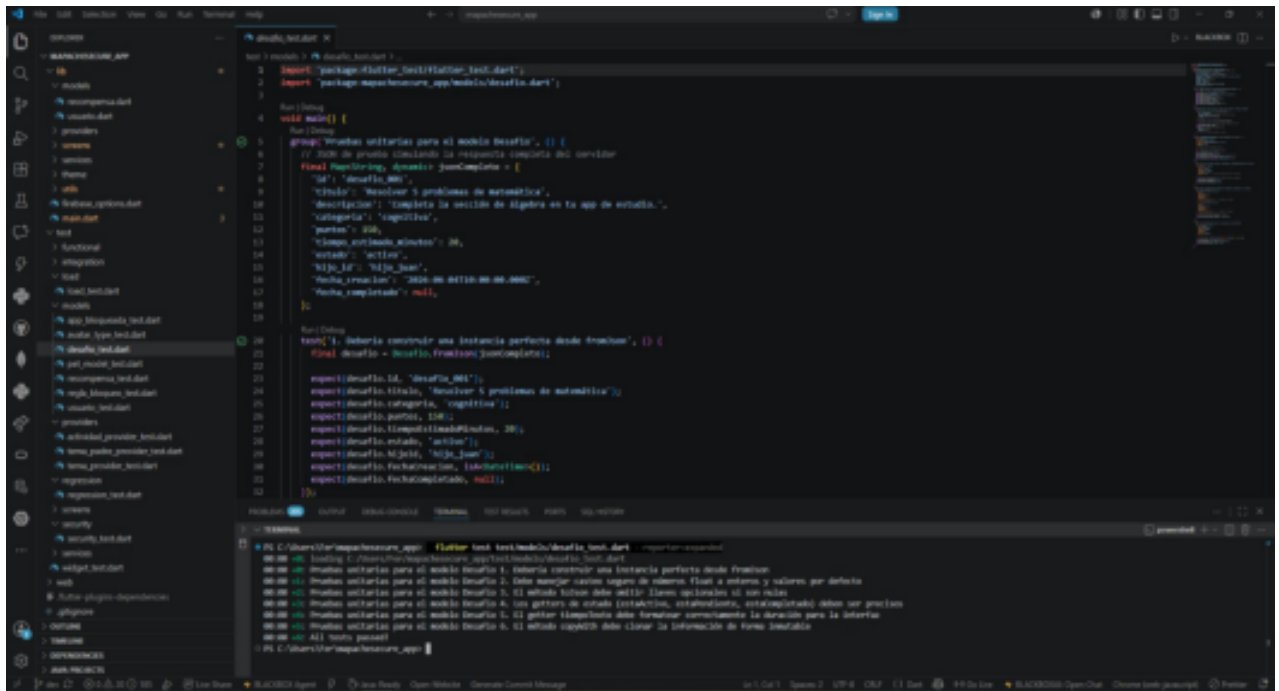
      // Assert: El primer elemento de la lista del mags es 'assets/sapachecore/mags/mags01.png'
      expect(avatar.previos, 'assets/sapachecore/mags/mags01.png');
    });

    // Test
    test('2. El método (magsIndex) debe calcular el índice correcto para niveles mags', () {
      // Act: Creamos una instancia de AvatarTypes
      final avatar = AvatarTypes.mags;

      // Assert: Nivel 1 -> Índice 0
      expect(avatar.magsIndex(1), 'assets/sapachecore/mags/mags01.png');
      // Assert: Nivel 2 -> Índice 1
      expect(avatar.magsIndex(2), 'assets/sapachecore/mags/mags02.png');
      // Assert: Nivel 3 -> Índice 2
      expect(avatar.magsIndex(3), 'assets/sapachecore/mags/mags03.png');
      // Assert: Nivel 4 -> Índice 3
      expect(avatar.magsIndex(4), 'assets/sapachecore/mags/mags04.png');
    });
  });
}

```

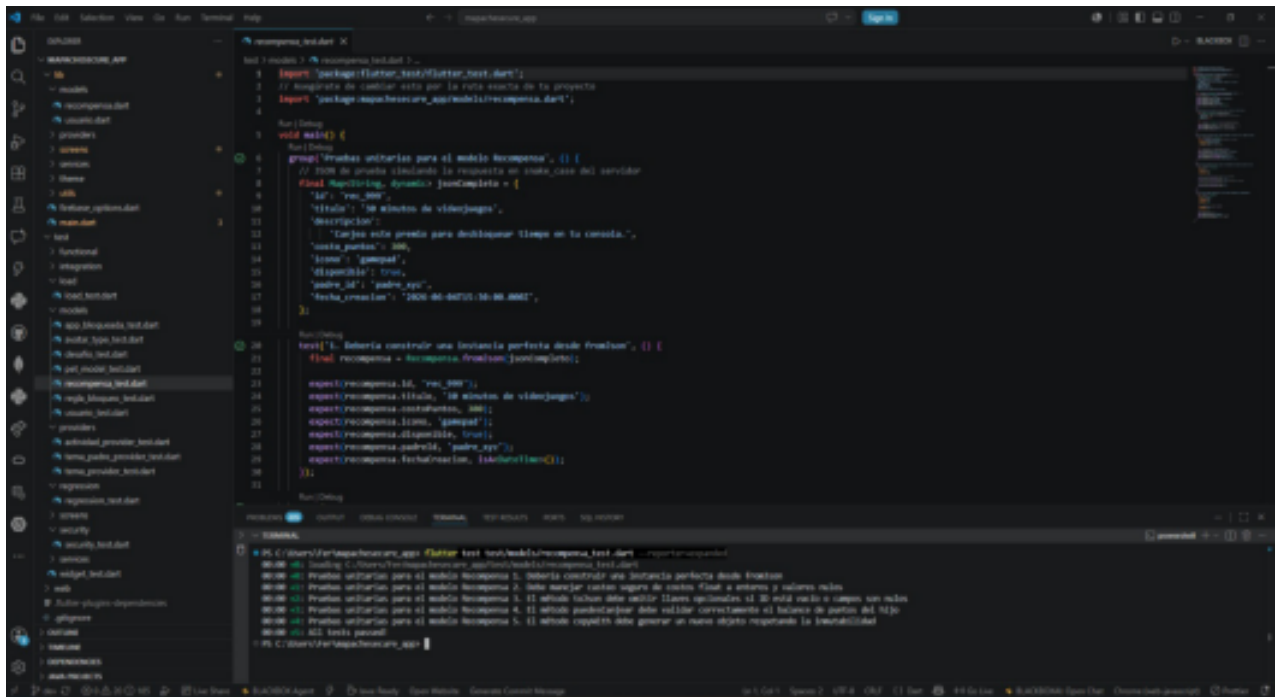
Avatar_type_test.dart



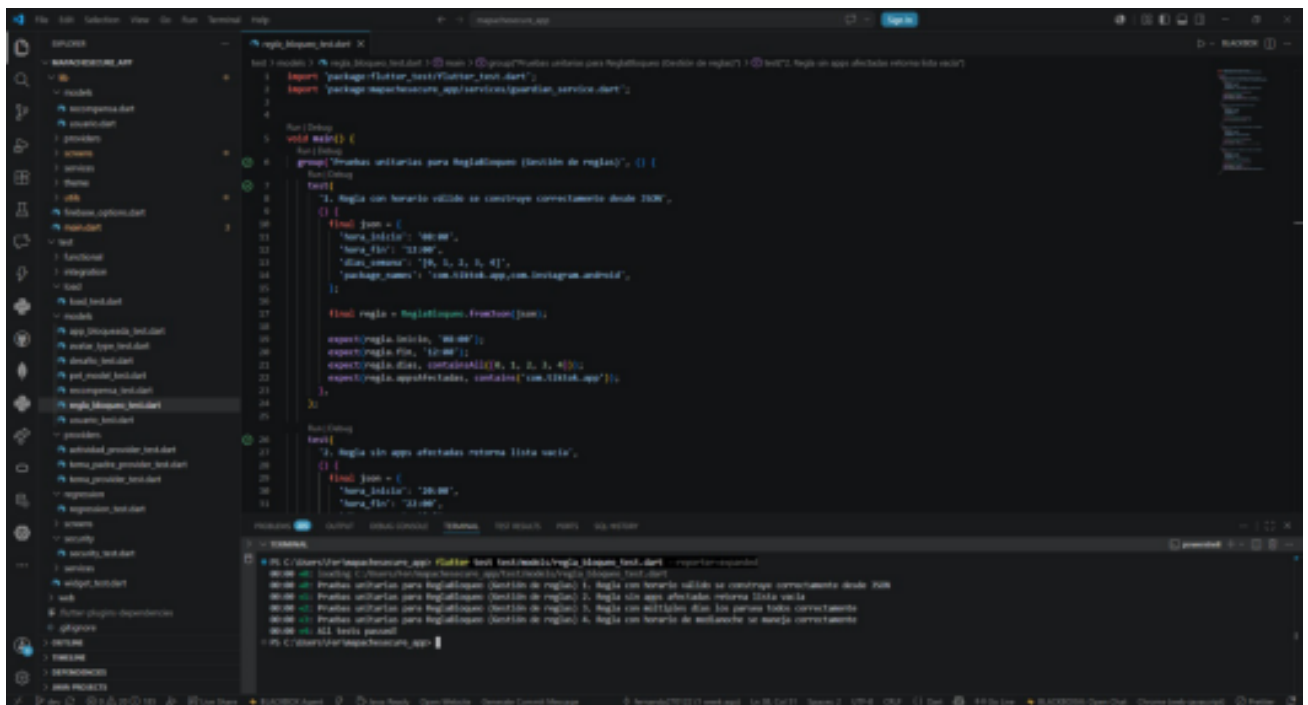
Desafio_test.dart



Pet_model_test.dart



Recompensa_test.dart



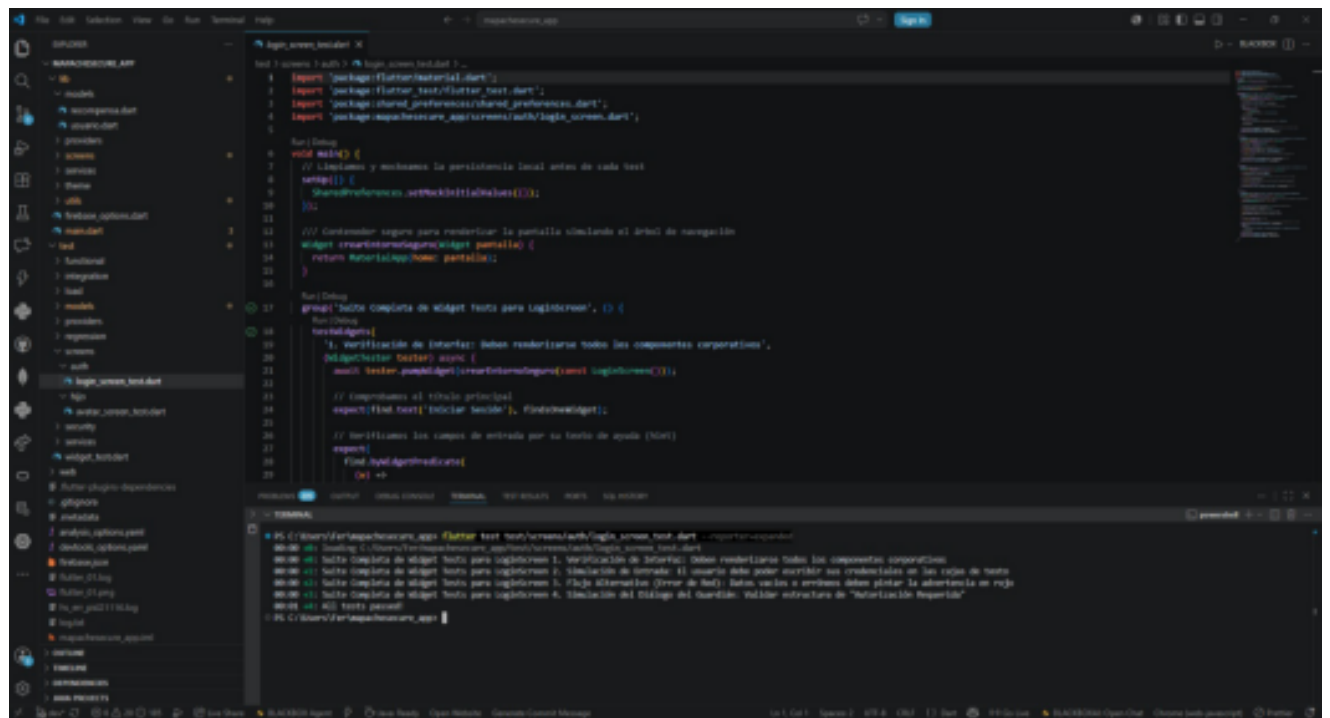
Regla_bloqueo_test.dart

[illegible]

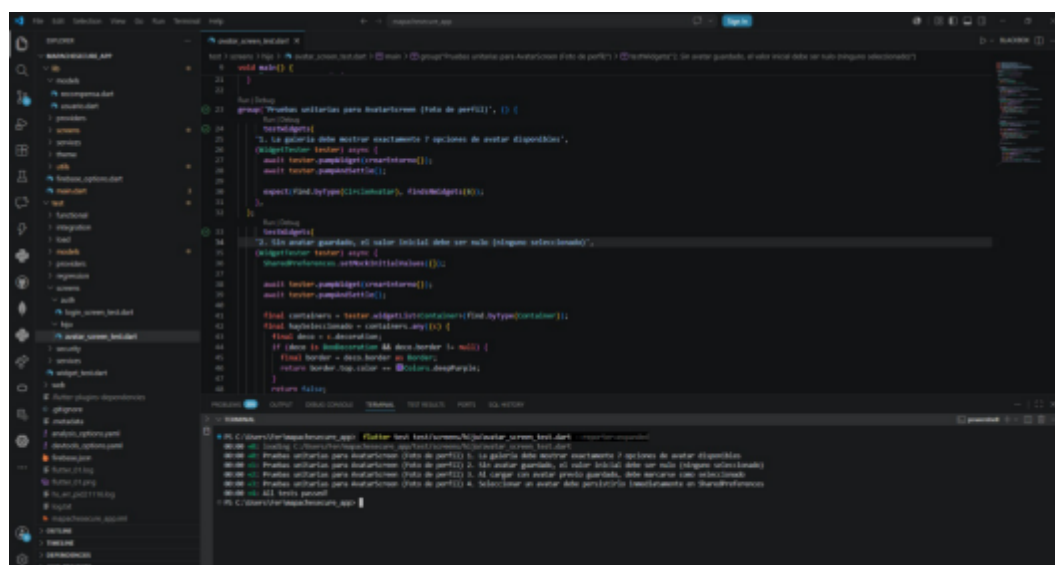
Actividad_provider_test.dart



Pantallas:

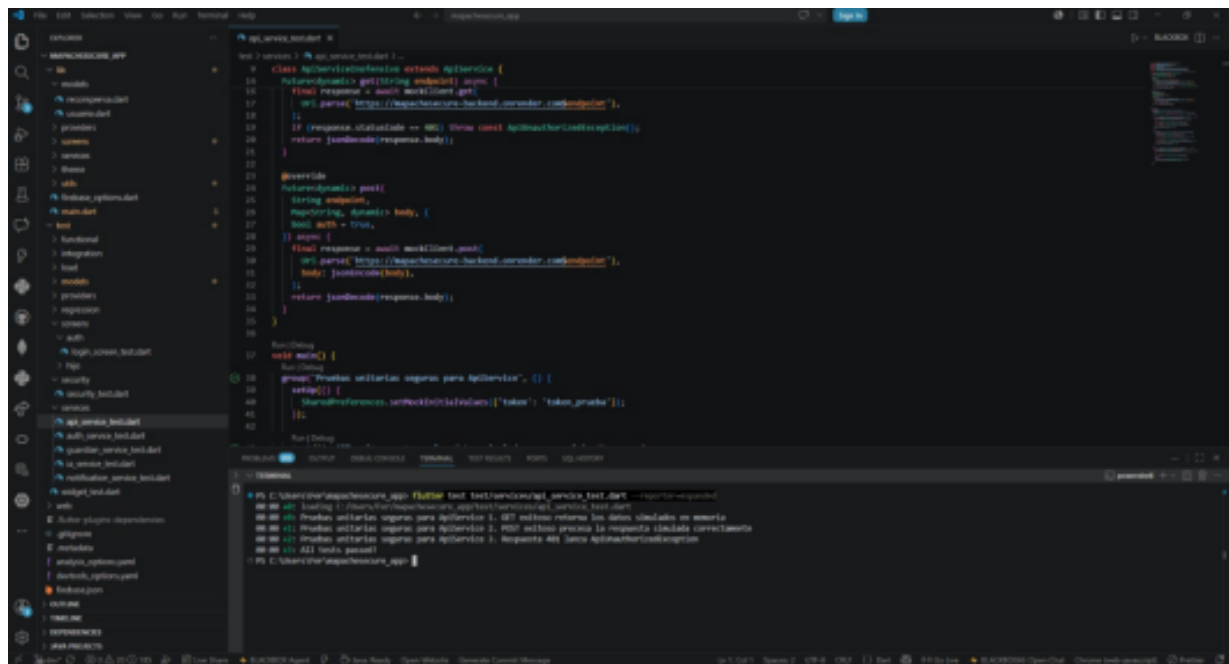


login_screen_test.dart

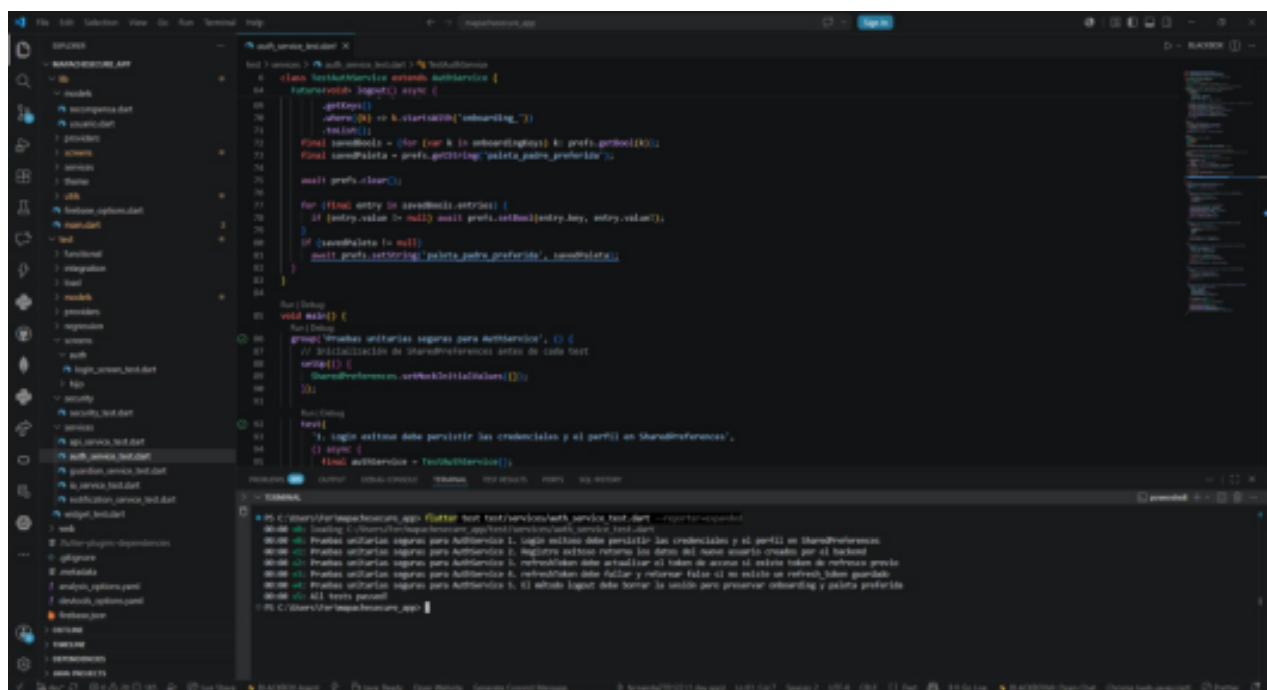


Avatar_screen_test.dart

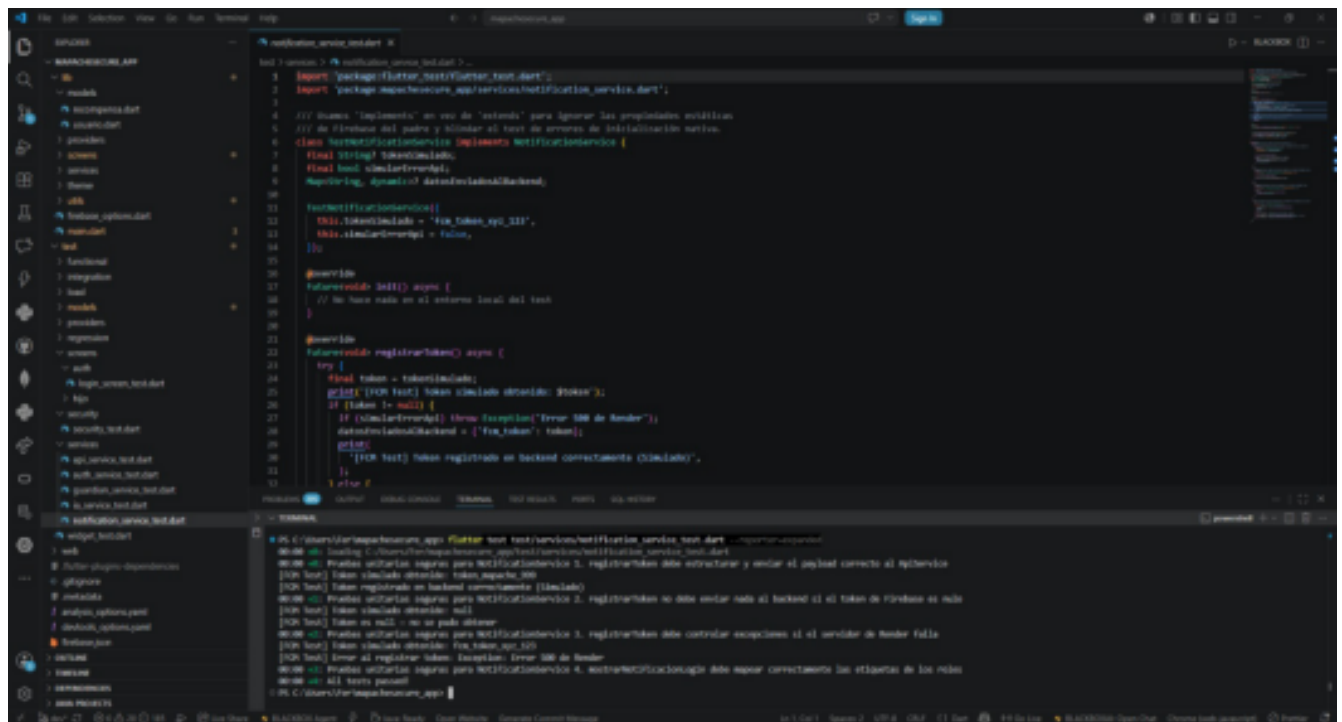
Services:



Api_service_test.dart



Auth_service_test.dart



Notification_service_test.dart

Pruebas de humo en dispositivo (9 casos)

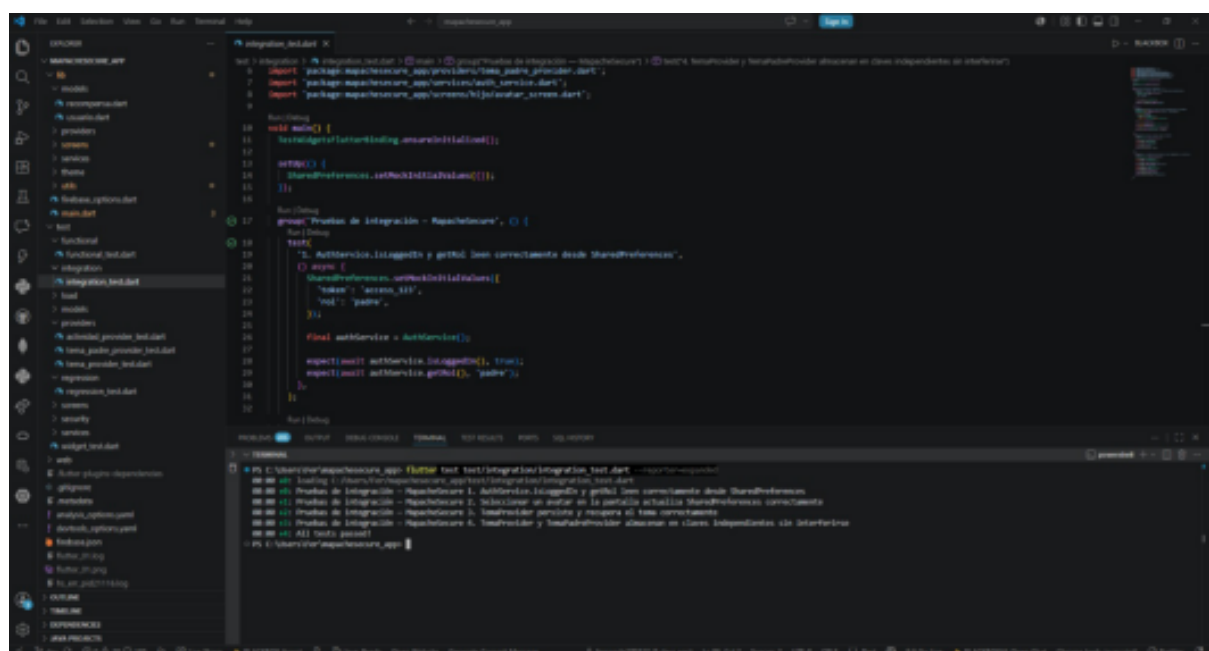
Ejecutadas en Vivo V2127 (Android 14, FuntouchOS) con flutter test integration_test/smoke_test.dart -d 3479446136003WM. Cada caso corresponde a una historia de usuario.

- HU-01 - RegistroScreen: formulario de registro arranca y renderiza campos. EXITOSO.
- HU-03 - LoginScreen: pantalla de login muestra campos de correo y contraseña. EXITOSO.
- HU-02 - AgregarHijoScreen: formulario de registro de hijo visible y funcional. EXITOSO.
- HU-08 - DesafiosHijoScreen: lista de desafíos del hijo carga con token del padre. EXITOSO.
- HU-04 - ConfigurarHijoScreen: pantalla de configuración de bloqueo renderizar correctamente. EXITOSO.
- HU-09 - RecompensasScreen: pantalla de recompensas muestra datos hardcoded. EXITOSO.
- HU-07 - MisDesafiosScreen: desafíos del hijo cargan con token real desde Supabase. EXITOSO.

- HU-05 - PantallaBloqueoScreen: muestra candado y horario; canPop=false impide salir. EXITOSO.

Pruebas de recuperación 4 casos

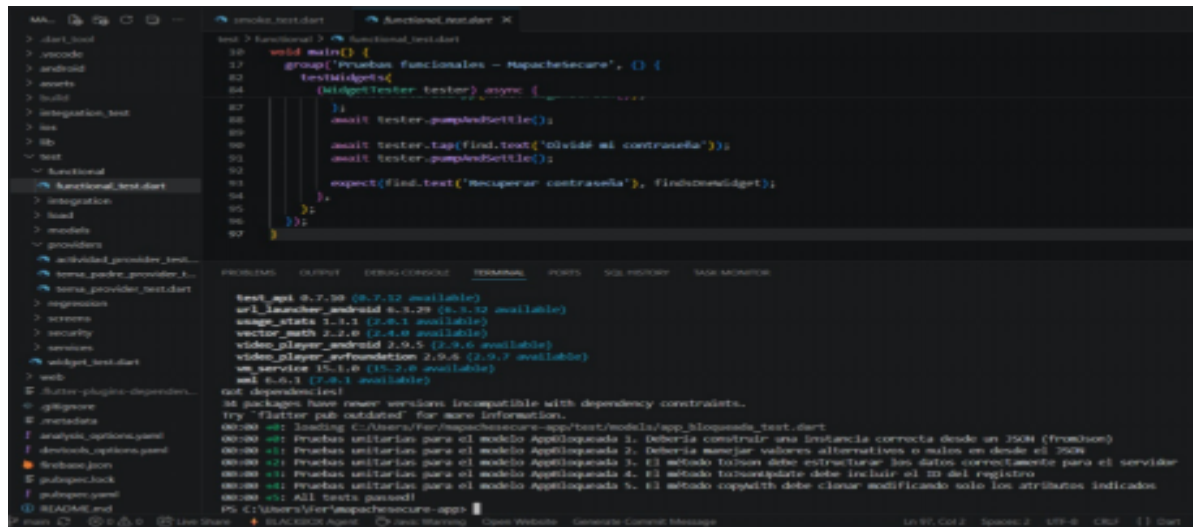
Verifican la recuperación entre: AuthService + SharedPreferences, persistencia de TemaProvider y TemaPadreProvider sin interferencia de claves, y



Integration_test.dart

Pruebas funcionales (4 casos)

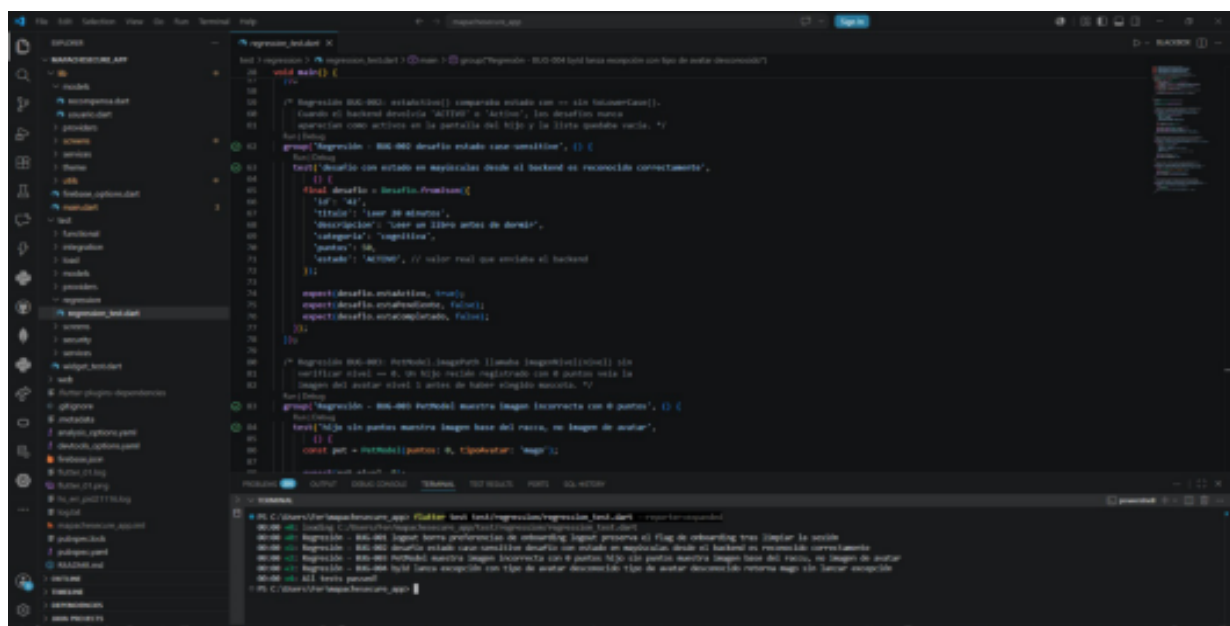
Login fallido con campo vacío muestra “Correo o contraseña incorrectos”, selección de avatar marca borde deepPurple; PantallaBloqueoScreen muestra horario correcto; tap en “Olvidé mi contraseña” navega a pantalla de recuperación. Resultado: 4/4 exitosos.



Functional_test.dart

Pruebas de regresión (4 casos)

Logout preserva onboarding_completado y paleta_padre_preferida; ID de avatar invalido retorna mago; PetModel no falla con puntos 0 ni 99999; estado "ACTIVO" en mayúsculas es reconocido como activo. Resultado: 4/4 exitosos.



Regression_test.dart

Pruebas de carga / rendimiento (4 casos)

Escenario	Umbral	Resultado
Parsear 1.000 desafíos desde JSON	< 500 ms	EXITOSO
Parsear 500 AppBloqueada desde JSON	< 300 ms	EXITOSO
Calcular tiempoTotalPantalla (100 registros)	< 200 ms	EXITOSO
10.000 búsquedas de avatar por ID	< 1.000 ms	EXITOSO

[illegible]

Load_test.dart

Pruebas de seguridad (4 casos)

Sin token -> isLoggedIn retorna false; eliminar token invalida la sesion; JSON con campos null no crashea Desafio.fromJson(); PopScope con canPop=false impide que el niño salga de la pantalla de bloqueo. Resultado: 4/4 exitosos.



Security_test.dart

Pruebas unitarias de backend (105 casos)

Ejecutadas el 14 de junio de 2026 con python -m pytest tests/ -v en el entorno virtual del proyecto mapachesecond-backend. Resultado: 105/105 en verde en 0.82 s.

- test_apps.py (16 casos): obtener, agregar, eliminar y actualizar apps bloqueadas; verificar estado y reportar uso diario.
- test_autenticacion.py (24 casos): registro, login, refresco de token, logout, vinculación de hijo, registro de hijo, recuperación y cambio de contraseña.
- test_canjes.py (6 casos): obtener pendientes, estado de canje por hijo, aprobar y rechazar canjes.
- test_desafios.py (22 casos): obtener, completar, validar y eliminar desafíos; obtener puntos y pendientes del padre.
- test_ia.py (7 casos): generar desafíos con IA, limpieza de bloques de código JSON, construcción correcta del prompt según perfil del niño.
- test_recompensas.py (16 casos): obtener, crear, canjear y actualizar recompensas; catálogo; historial de canjes.

- [illegible]

The screenshot shows the PyCharm IDE interface. On the left, the 'Project Explorer' pane lists the files and folders of the 'Proyecto' project. The main window displays the contents of 'test_usuario.py'. The code consists of several test functions, each starting with a docstring indicating the purpose of the test and the expected outcome.

```

def test_usuario_crear():
    """Test de crear usuario"""
    # ... (code omitted) ...

def test_usuario_eliminar():
    """Test de eliminar usuario"""
    # ... (code omitted) ...

def test_usuario_actualizar():
    """Test de actualizar usuario"""
    # ... (code omitted) ...

def test_usuario_buscar():
    """Test de buscar usuario"""
    # ... (code omitted) ...

def test_usuario_listar():
    """Test de listar usuarios"""
    # ... (code omitted) ...

def test_usuario_validar_credenciales():
    """Test de validar credenciales"""
    # ... (code omitted) ...

def test_usuario_resetear_password():
    """Test de resetear password"""
    # ... (code omitted) ...

def test_usuario_verificar_token():
    """Test de verificar token"""
    # ... (code omitted) ...

def test_usuario_obtener_perfil():
    """Test de obtener perfil de usuario"""
    # ... (code omitted) ...

def test_usuario_loguearse():
    """Test de loguearse"""
    # ... (code omitted) ...

def test_usuario_registrarse():
    """Test de registrarse"""
    # ... (code omitted) ...

def test_usuario_logout():
    """Test de logout"""
    # ... (code omitted) ...

def test_usuario_login_fallido():
    """Test de login fallido"""
    # ... (code omitted) ...

def test_usuario_registro_fallido():
    """Test de registro fallido"""
    # ... (code omitted) ...

def test_usuario_recuperacion_password():
    """Test de recuperación de password"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido():
    """Test de verify token fallido"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido():
    """Test de get profile fallido"""
    # ... (code omitted) ...

def test_usuario_login_exitoso():
    """Test de login exitoso"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso():
    """Test de registro exitoso"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa():
    """Test de recuperación exitosa"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso():
    """Test de verify token exitoso"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso():
    """Test de get profile exitoso"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso():
    """Test de logout exitoso"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_credencial_invalida():
    """Test de login fallido por credencial invalida"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_email_existente():
    """Test de registro fallido por email existente"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_email_no_existe():
    """Test de recuperación fallida por email no existe"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token_invalido():
    """Test de verify token fallido por token invalido"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario_no_encontrado():
    """Test de get profile fallido por usuario no encontrado"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_con_jwt_token():
    """Test de login exitoso con JWT token"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_con_confirmacion():
    """Test de registro exitoso con confirmación"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_con_codigo_verificacion():
    """Test de recuperación exitosa con código de verificación"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_con_token_valido():
    """Test de verify token exitoso con token valido"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_con_usuario_valido():
    """Test de get profile exitoso con usuario valido"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token_valido():
    """Test de logout exitoso con token valido"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_usuario_inactivo():
    """Test de login fallido por usuario inactivo"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_password_muy debil():
    """Test de registro fallido por password muy debil"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_password_nuevo_muy debil():
    """Test de recuperación fallida por password nuevo muy debil"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token_expired():
    """Test de verify token fallido por token expired"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario_banado():
    """Test de get profile fallido por usuario banado"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_después_de_recuperacion():
    """Test de login exitoso después de recuperación"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_después_de_registro():
    """Test de registro exitoso después de registro"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_después_de_recuperacion():
    """Test de recuperación exitosa después de recuperación"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_después_de_login():
    """Test de verify token exitoso después de login"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_después_de_login():
    """Test de get profile exitoso después de login"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_después_de_login():
    """Test de logout exitoso después de login"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_usuario_inexistente():
    """Test de login fallido por usuario inexistente"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_email_incorrecto():
    """Test de registro fallido por email incorrecto"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_email_incorrecto():
    """Test de recuperación fallida por email incorrecto"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token_incorrecto":
    """Test de verify token fallido por token incorrecto"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario_incorrecto":
    """Test de get profile fallido por usuario incorrecto"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_sin_token():
    """Test de logout exitoso sin token"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_con_password largo():
    """Test de login exitoso con password largo"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_con_password corto():
    """Test de registro exitoso con password corto"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_con_password largo():
    """Test de recuperación exitosa con password largo"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_con_token largo():
    """Test de verify token exitoso con token largo"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_con_usuario largo():
    """Test de get profile exitoso con usuario largo"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token largo():
    """Test de logout exitoso con token largo"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_usuario muy largo():
    """Test de login fallido por usuario muy largo"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_email muy largo():
    """Test de registro fallido por email muy largo"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_email muy largo():
    """Test de recuperación fallida por email muy largo"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token muy largo():
    """Test de verify token fallido por token muy largo"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario muy largo():
    """Test de get profile fallido por usuario muy largo"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy largo():
    """Test de logout exitoso con token muy largo"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_con_password muy largo():
    """Test de login exitoso con password muy largo"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_con_password muy largo():
    """Test de registro exitoso con password muy largo"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_con_password muy largo():
    """Test de recuperación exitosa con password muy largo"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_con_token muy largo():
    """Test de verify token exitoso con token muy largo"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_con_usuario muy largo():
    """Test de get profile exitoso con usuario muy largo"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy largo():
    """Test de logout exitoso con token muy largo"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_usuario muy corto():
    """Test de login fallido por usuario muy corto"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_email muy corto():
    """Test de registro fallido por email muy corto"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_email muy corto():
    """Test de recuperación fallida por email muy corto"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token muy corto":
    """Test de verify token fallido por token muy corto"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario muy corto":
    """Test de get profile fallido por usuario muy corto"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy corto":
    """Test de logout exitoso con token muy corto"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_con_password muy corto":
    """Test de login exitoso con password muy corto"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_con_password muy corto":
    """Test de registro exitoso con password muy corto"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_con_password muy corto":
    """Test de recuperación exitosa con password muy corto"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_con_token muy corto":
    """Test de verify token exitoso con token muy corto"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_con_usuario muy corto":
    """Test de get profile exitoso con usuario muy corto"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy corto":
    """Test de logout exitoso con token muy corto"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_usuario muy especial():
    """Test de login fallido por usuario muy especial"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_email muy especial":
    """Test de registro fallido por email muy especial"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_email muy especial":
    """Test de recuperación fallida por email muy especial"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token muy especial":
    """Test de verify token fallido por token muy especial"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario muy especial":
    """Test de get profile fallido por usuario muy especial"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy especial":
    """Test de logout exitoso con token muy especial"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_con_password muy especial":
    """Test de login exitoso con password muy especial"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_con_password muy especial":
    """Test de registro exitoso con password muy especial"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_con_password muy especial":
    """Test de recuperación exitosa con password muy especial"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_con_token muy especial":
    """Test de verify token exitoso con token muy especial"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_con_usuario muy especial":
    """Test de get profile exitoso con usuario muy especial"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy especial":
    """Test de logout exitoso con token muy especial"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_usuario muy aleatorio":
    """Test de login fallido por usuario muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_email muy aleatorio":
    """Test de registro fallido por email muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_email muy aleatorio":
    """Test de recuperación fallida por email muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token muy aleatorio":
    """Test de verify token fallido por token muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario muy aleatorio":
    """Test de get profile fallido por usuario muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy aleatorio":
    """Test de logout exitoso con token muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_con_password muy aleatorio":
    """Test de login exitoso con password muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_con_password muy aleatorio":
    """Test de registro exitoso con password muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_con_password muy aleatorio":
    """Test de recuperación exitosa con password muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_con_token muy aleatorio":
    """Test de verify token exitoso con token muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_con_usuario muy aleatorio":
    """Test de get profile exitoso con usuario muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy aleatorio":
    """Test de logout exitoso con token muy aleatorio"""
    # ... (code omitted) ...

def test_usuario_login_fallido_por_usuario muy unico":
    """Test de login fallido por usuario muy unico"""
    # ... (code omitted) ...

def test_usuario_registro_fallido_por_email muy unico":
    """Test de registro fallido por email muy unico"""
    # ... (code omitted) ...

def test_usuario_recuperacion_fallida_por_email muy unico":
    """Test de recuperación fallida por email muy unico"""
    # ... (code omitted) ...

def test_usuario_verify_token_fallido_por_token muy unico":
    """Test de verify token fallido por token muy unico"""
    # ... (code omitted) ...

def test_usuario_get_profile_fallido_por_usuario muy unico":
    """Test de get profile fallido por usuario muy unico"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy unico":
    """Test de logout exitoso con token muy unico"""
    # ... (code omitted) ...

def test_usuario_login_exitoso_con_password muy unico":
    """Test de login exitoso con password muy unico"""
    # ... (code omitted) ...

def test_usuario_registro_exitoso_con_password muy unico":
    """Test de registro exitoso con password muy unico"""
    # ... (code omitted) ...

def test_usuario_recuperacion_exitosa_con_password muy unico":
    """Test de recuperación exitosa con password muy unico"""
    # ... (code omitted) ...

def test_usuario_verify_token_exitoso_con_token muy unico":
    """Test de verify token exitoso con token muy unico"""
    # ... (code omitted) ...

def test_usuario_get_profile_exitoso_con_usuario muy unico":
    """Test de get profile exitoso con usuario muy unico"""
    # ... (code omitted) ...

def test_usuario_logout_exitoso_con_token muy unico":
    """Test de logout exitoso con token muy unico"""
    #
```

```
tests/test_usuario.py::testConfigurarPerfil::test_configurar_actualiza_el_tiempo_limite PASSED [ 54%]
tests/test_usuario.py::testConfigurarPerfil::test_configurar_con_error_desuave_500 PASSED [ 55%]
tests/test_usuario.py::testActualizarAvatar::test_avatar_valido_se_actualiza PASSED [ 56%]
tests/test_usuario.py::testActualizarAvatar::test_avatar_invalido_desuave_400 PASSED [ 57%]
tests/test_usuario.py::testActualizarAvatar::test_todo_los_avatares_validos_se aceptan PASSED [ 58%]
tests/test_usuario.py::testActualizarPerfil::test_actualiza_la foto de perfil PASSED [ 59%]
tests/test_usuario.py::testActualizarPerfil::test_con_error_desuave_500 PASSED [ 60%]

----- 906 passed in 0.85s -----
```

Test del Backend

Conclusión 1

MapacheSecure nace como respuesta a un problema real y creciente: los niños pasan cada vez más tiempo frente a las pantallas, sin herramientas que les ayuden a autorregularse de manera positiva. Las soluciones existentes se limitan a bloquear o restringir, pero ninguna propone una experiencia que haga del autocontrol algo motivador para el niño y manejable para el padre.

La solución propuesta combina un sistema de bloqueo efectivo con actividades de reemplazo y un motor de recompensas personalizable, construido sobre una arquitectura sólida y alcanzable para el equipo: una app Android en Kotlin, un backend en Python con FastAPI, una base de datos en Supabase conectada mediante SQLAlchemy, y notificaciones a través de Firebase Cloud Messaging.

Los beneficios esperados son cuatro: reducción del tiempo de pantalla sin conflicto familiar, desarrollo de hábitos positivos en los niños, mayor control y visibilidad para los padres, y una experiencia que convierte el límite en un juego en lugar de un castigo.

Como próximos pasos, el equipo continuará con el levantamiento detallado de requerimientos, la construcción del prototipo de la interfaz y la configuración del entorno de desarrollo compartido en GitHub, dando inicio formal a la fase de construcción del sistema.

fecha: 24-05-2026

Este informe presenta el estado de desarrollo de MapacheSecure al cierre del Sprint 2. A lo largo del documento se expuso la metodología de trabajo basada en Scrum, los diagramas de diseño del sistema, la arquitectura implementada y el stack tecnológico definitivo.

El equipo logró implementar las funcionalidades core del sistema: autenticación, gestión de usuarios, módulo de desafíos, sistema de recompensas, bloqueo de aplicaciones, avatar con evolución por puntos y personalización de perfil. Todo el backend está desplegado y funcionando en producción sobre Render y Supabase.

Como próximos pasos para el Sprint 3, el equipo se enfocará en completar la integración de notificaciones push con Firebase FCM, implementar el módulo de generación adaptativa de desafíos con IA, realizar pruebas de integración completas y preparar el entregable final del sistema.

fecha: 18-06-2026

Raccu nació como respuesta a la ausencia de una solución de control parental pensada para niños de 5 a 15 años que combine bloqueo efectivo, gamificación e inteligencia artificial. A lo largo del Sprint 3 (26 mayo - 30 junio de 2026) el equipo implementó y ejecutó un plan de pruebas completo que cubre las siete dimensiones de calidad exigidas por la pauta.

Beneficios obtenidos:

- Confianza en la funcionalidad central: 210 pruebas automatizadas en verde garantizan que los módulos clave del frontend y backend (bloqueo, desafíos, mascota, autenticación, recompensas, IA) funcionan correctamente.
- Cobertura end-to-end: el frontend Flutter y el backend FastAPI tienen pruebas independientes, lo que permite detectar errores en cualquiera de las dos capas.
- Seguridad validada: la sesión se invalida correctamente y la pantalla de bloqueo es infranqueable desde el dispositivo del niño.
- Rendimiento verificado: el procesamiento de datos en memoria cumple umbrales estrictos, asegurando una experiencia fluida incluso con grandes volúmenes.
- Regresión controlada: el equipo puede realizar cambios con la certeza de que comportamientos anteriores no se romperán silenciosamente.

Conclusión General

MapacheSecure — Raccu recorrió un camino completo en tres etapas: desde una propuesta conceptual con tecnologías por definir, pasando por la construcción de todas las funcionalidades core del sistema, hasta la validación formal mediante 210 pruebas automatizadas que cubrieron el frontend Flutter, el backend FastAPI y pruebas en dispositivo físico real.

El resultado es una aplicación funcional, probada y desplegada en producción, que cumple su propósito original: ofrecer a los padres una herramienta de control parental que combina bloqueo efectivo, gamificación e inteligencia artificial, convirtiendo el límite de pantalla en una experiencia positiva para el niño.

Referencias Bibliográficas

- Forest: Stay Focused. (2026). Aplicación de gamificación y productividad. Recuperado de: <https://www.forestapp.cc/>
- Freedom. (2026). Bloqueador de distracciones multiplataforma. Recuperado de: <https://freedom.to/>
- Opal: Screen Time for Focus. (2026). Herramienta de bienestar digital. Recuperado de: <https://opalapp.com/>
- Stay Focused. (2026). Extensión y aplicación de control de tiempo en sitios web. Recuperado de: <https://www.stayfocusd.com/>
- Docta Complutense. (s.f.). Análisis de aplicaciones para la gestión del tiempo y adicción al smartphone. Recuperado de: <https://docta.ucm.es/entities/publication/c7df0c57-cefb-4fdc-a051-e9e29b729051>

Anexos

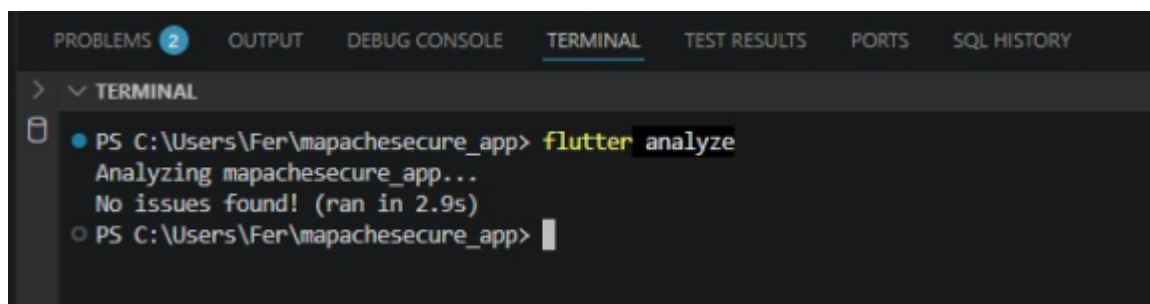
Análisis estático de código — flutter analyze (Linter)

flutter analyze es el linter oficial de Flutter/Dart, equivalente a ESLint en JavaScript, pylint en Python o RuboCop en Ruby. Un linter es una herramienta de análisis estático que revisa el código fuente sin ejecutarlo, detectando errores potenciales, uso de APIs obsoletas, malas prácticas y violaciones de estilo. En proyectos profesionales, pasar el linter con 0 advertencias es un requisito habitual antes de aprobar un Pull Request.

Se ejecutó flutter analyze sobre el proyecto mapacheseccure_app una vez finalizadas las correcciones. El análisis reportó 0 issues, partiendo de un estado inicial de 138 advertencias (0 errores, 0 warnings, 138 infos). Las categorías corregidas fueron:

- deprecated_member_use: withOpacity() reemplazado por withValues(alpha:) en 30 archivos; activeColor reemplazado por activeThumbColor en 5 pantallas; value reemplazado por initialValue en DropdownButtonFormField.
- avoid_print: print() reemplazado por debugPrint() en 8 archivos (solo emite en modo debug).
- use_build_context_synchronously: 13 usos de BuildContext tras await protegidos con if (context.mounted).
- curly_braces_in_flow_control_structures: llaves añadidas a 17 sentencias if sin bloque.
- unnecessary_underscores: parametros __ y ___ reemplazados por _ (comodin Dart 3+).
- use_null_aware_elements: patron if (x!=null) reemplazado por operador ?.
- unnecessary_import: import redundante de foundation.dart eliminado.
- file_names: Actividad_hijo_screen.dart renombrado a actividad_hijo_screen.dart.

Resultado final:



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS SQL HISTORY
> v TERMINAL
• PS C:\Users\Fer\mapacheseccure_app> flutter analyze
Analyzing mapacheseccure_app...
No issues found! (ran in 2.9s)
○ PS C:\Users\Fer\mapacheseccure_app> |
```